JETSON AI BPLTV-KPM WORKSHOP

JETSON, GIT, ANACONDA & PYTHON PROGRAMMING (PART 2)

PREPARED BY:
Center for Artificial Intelligence and Robotics
Universiti Teknologi Malaysia

Contents

WHAT IS GitHub?

WHAT IS GitHub?

CAIRO UTM

- Definition: GitHub is a web-based platform for version control and collaboration, built on top of Git.
- Purpose: It allows multiple people to work on a project simultaneously, track changes, and manage code repositories efficiently.
- Features: Hosting for Git repositories, collaboration tools, issue tracking, continuous integration and deployment (CI/CD) with GitHub Actions, and more.

BENEFITS OF USING GitHub

1. **Building a Professional Portfolio**
    - **Showcase Your Work:** Use GitHub repositories to showcase projects, demonstrating your coding skills and project management abilities to potential employers.
    - **Contributions:** Highlight contributions to open-source projects, showing collaboration and teamwork skills.
2. **Learning from Open-Source Projects**
    - **Access to Code:** Explore and learn from the code of experienced developers by browsing popular open-source repositories.
    - **Contributing:** Improve your skills by contributing to open-source projects, which also helps build your reputation in the developer community.

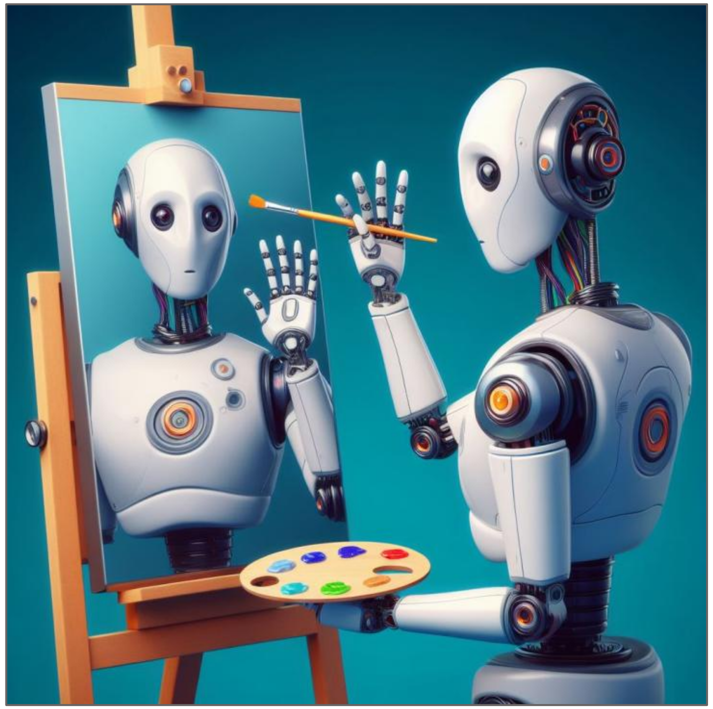
BENEFITS OF USING GitHub

3. **Collaboration and Teamwork**
    - **Working in Teams:** GitHub is widely used in professional settings for collaborative work. Learning to use it effectively prepares you for teamwork in real-world projects.
    - **Feedback and Code Reviews:** Engage in code reviews and discussions, receiving feedback and improving your code quality.
4. **Project Management Skills**
    - **Issues and Project Boards:** Manage tasks, bugs, and project milestones using GitHub Issues and Project Boards, enhancing your project management skills.
    - **Workflow Automation:** Use GitHub Actions to automate repetitive tasks, learning about CI/CD pipelines and improving productivity.

BENEFITS OF USING GitHub

5. Continuous Learning

- **Staying Updated:** Follow repositories and developers to stay updated on the latest technologies and trends in the software development industry.
- **GitHub Education:** Take advantage of resources and tools provided by GitHub Education for students.



UNDERSTANDING GIT & GitHub

Git: A Version Control System

- Git is a distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Key Features

Distributed: Every developer has a full copy of the project history.

Branching and Merging: Support for nonlinear development through branches.

Performance: Optimized for high performance.

Security: Ensures the integrity of the code and history.

Workflow

Working directory → **Staging Area** → **Repository**

Working Directory: The files in your project.

Staging Area: A place to store changes before committing them.

Repository: The history of all commits and branches.

UNDERSTANDING GIT & GitHub

GitHub: A Cloud-Based Hosting Service for Git Repositories

- GitHub is a platform for hosting and collaborating on Git repositories.

GitHub vs. Other Git Hosting Services:

- GitHub: Known for its user-friendly interface, community engagement, and GitHub Actions for Continuous Integration (CI)/ Continuous Deployment/Delivery(CD).
- Alternatives: GitLab, Bitbucket, and others, each with unique features and strengths.

Key features

1. Collaboration**:** Tools for code review, issue tracking, and project management.
2. Integration: Supports integration with CI/CD tools and other development tools.
3. Community: Millions of developers and repositories available for collaboration and learning.

RELATIONSHIP OF GIT & GitHub

- **Git:**
  Tool: Command-line tool installed locally on your computer.
  Function: Version control system to track changes in your project.

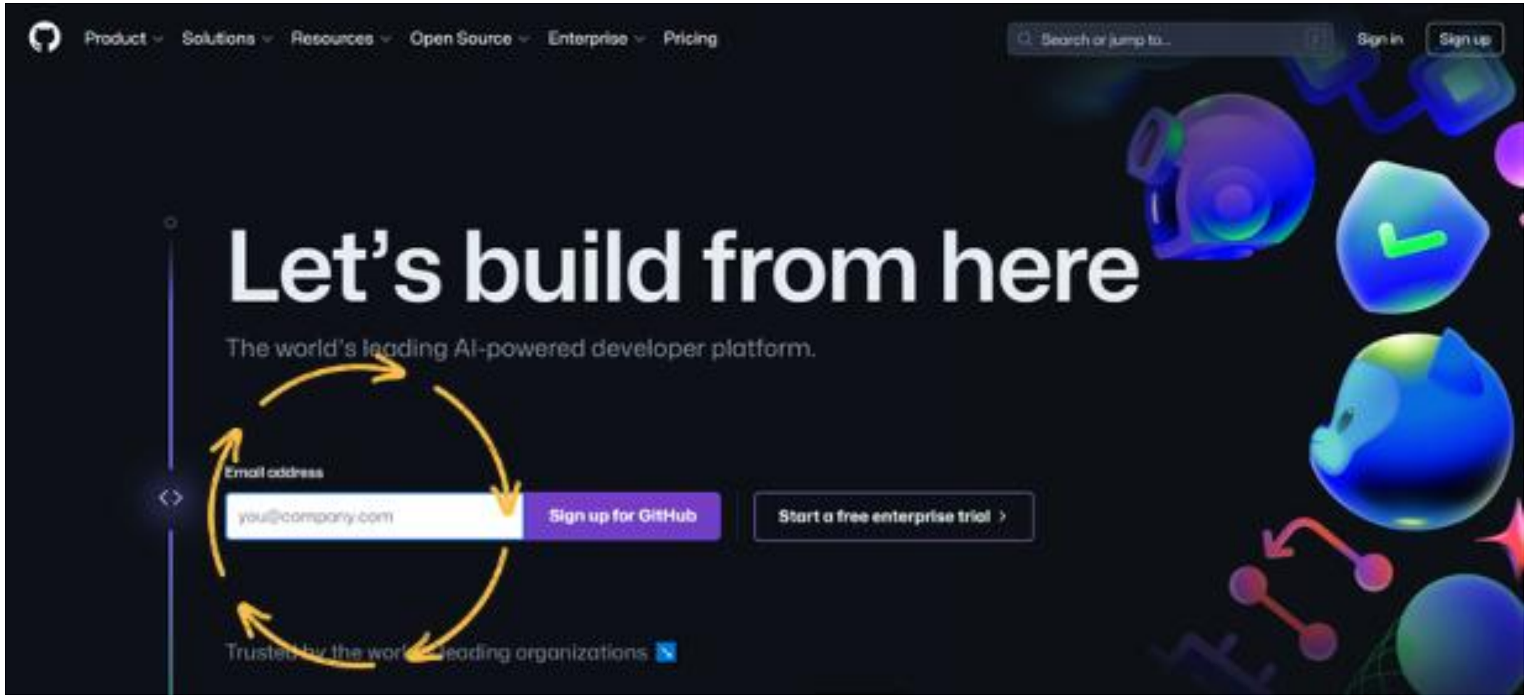
- **GitHub:**
  Service: Online platform that hosts Git repositories.
  Function: Facilitates collaboration, code sharing, and project management.

- **Relationship:**
  Workflow: Developers use Git to track changes locally and push/pull changes to/from GitHub for collaboration.

SETTING UP GitHub

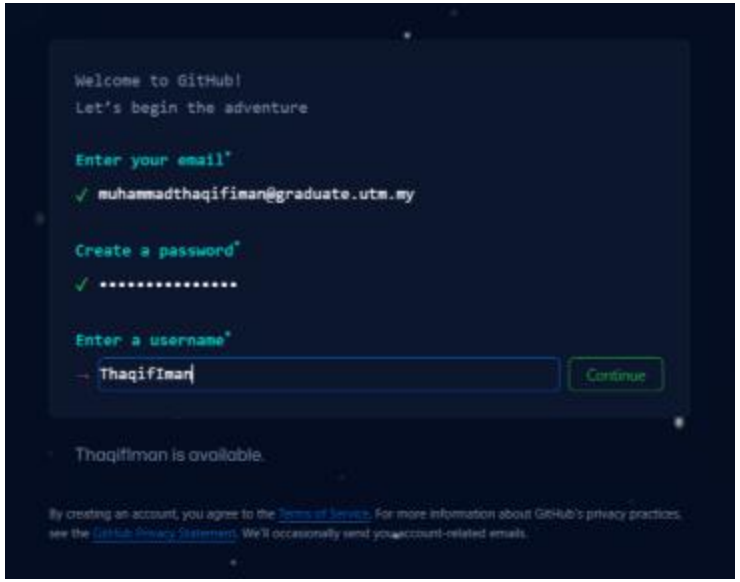
Creating a GitHub Account:

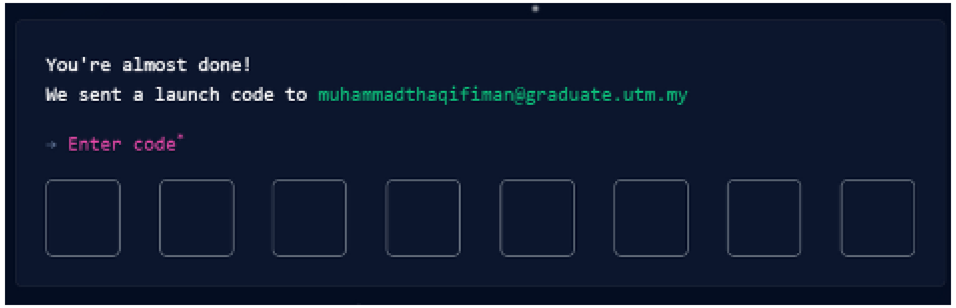
Sign Up: Go to GitHub's website and enter your username, email, and password. Click “SIGN UP”. Follow the prompts to complete the registration.



SETTING UP GitHub

Email Verification: Verify your email address by clicking the link sent to your inbox.





CREATING AND MANAGING REPOSITORIES

1. Creating a New Repository

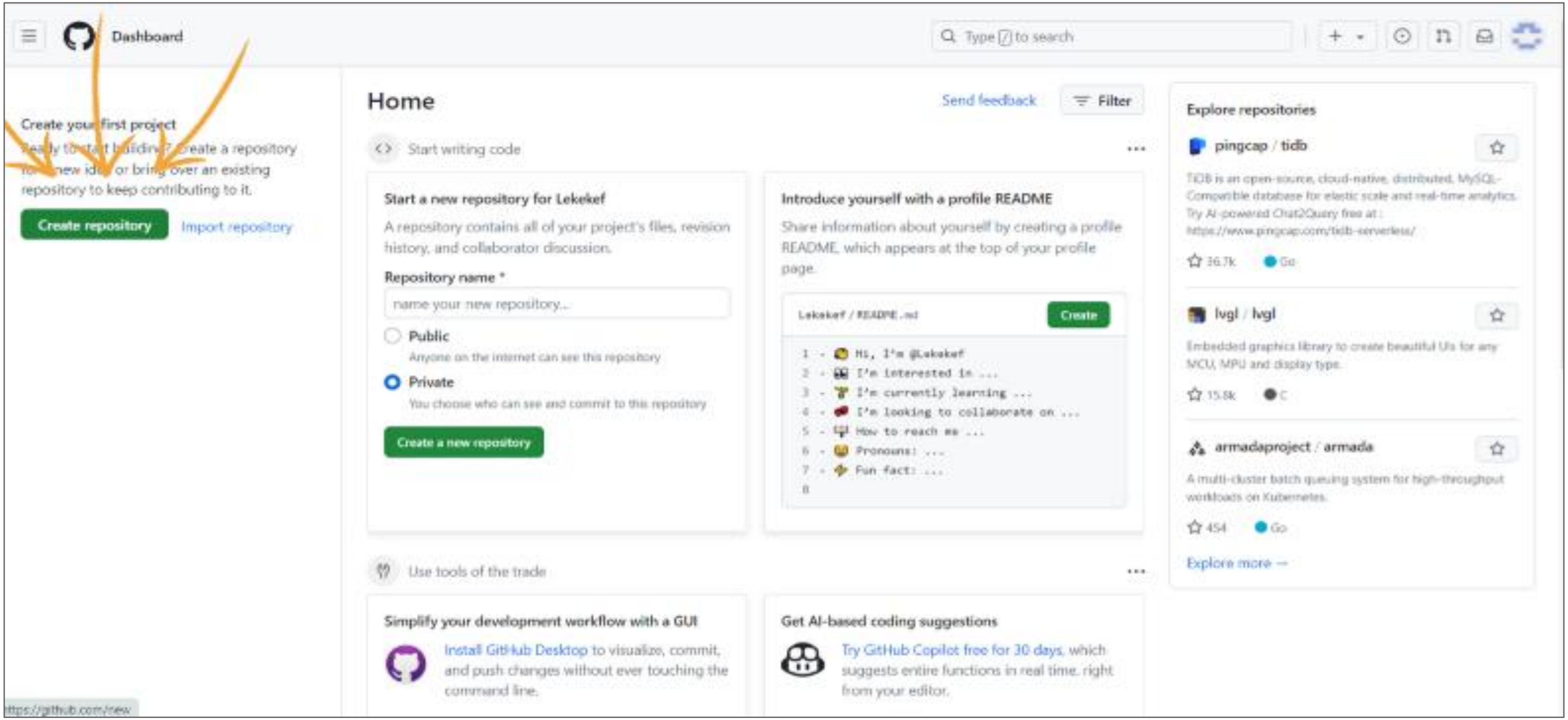
Steps:

- Navigate to Repositories: Click on the “Create Repositories" tab in your GitHub page.

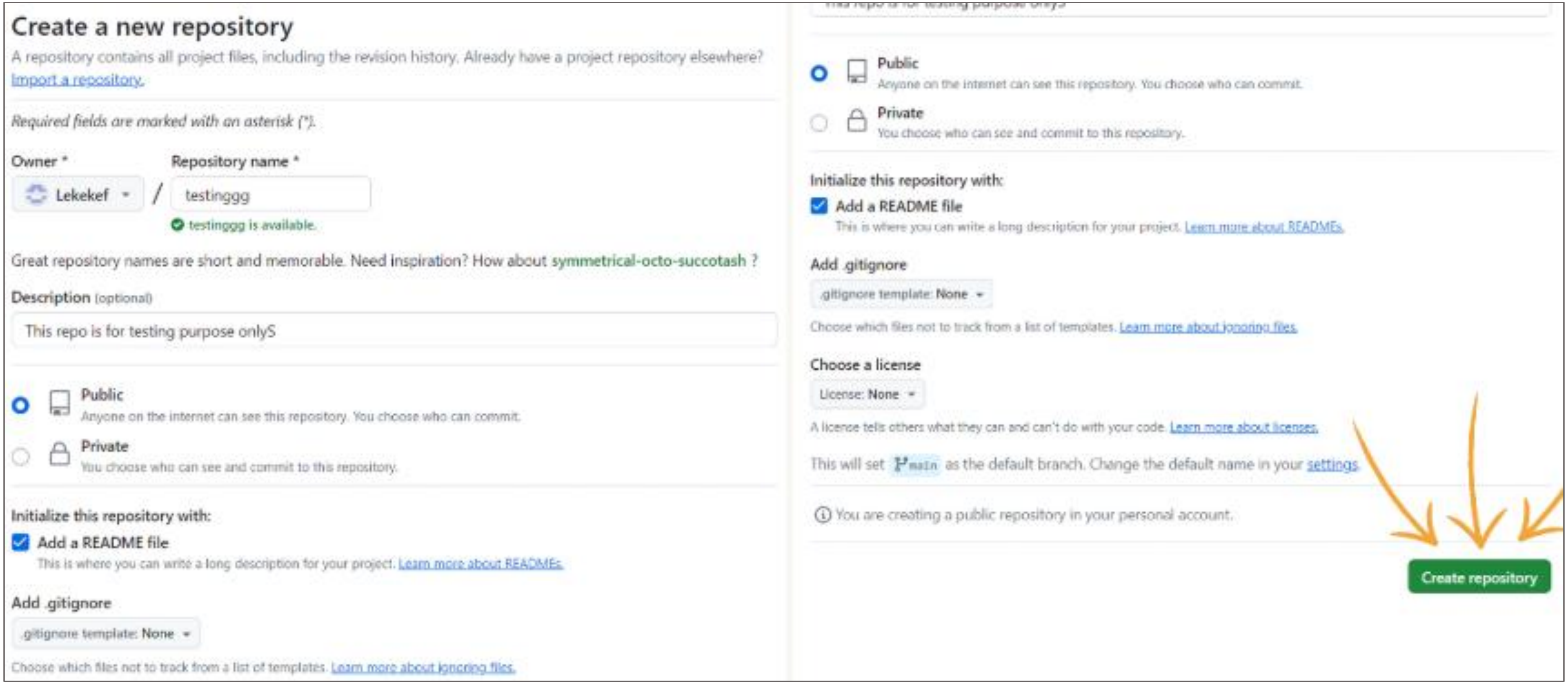
Fill in Details:

- Repository Name: Choose a unique name for your repository.
- Description: Optionally, add a description of your repository.
- Public/Private: Choose whether the repository will be public or private.
- Initialize with a README: Check this box to add a README file.
- Create Repository: Click the "Create repository" button.

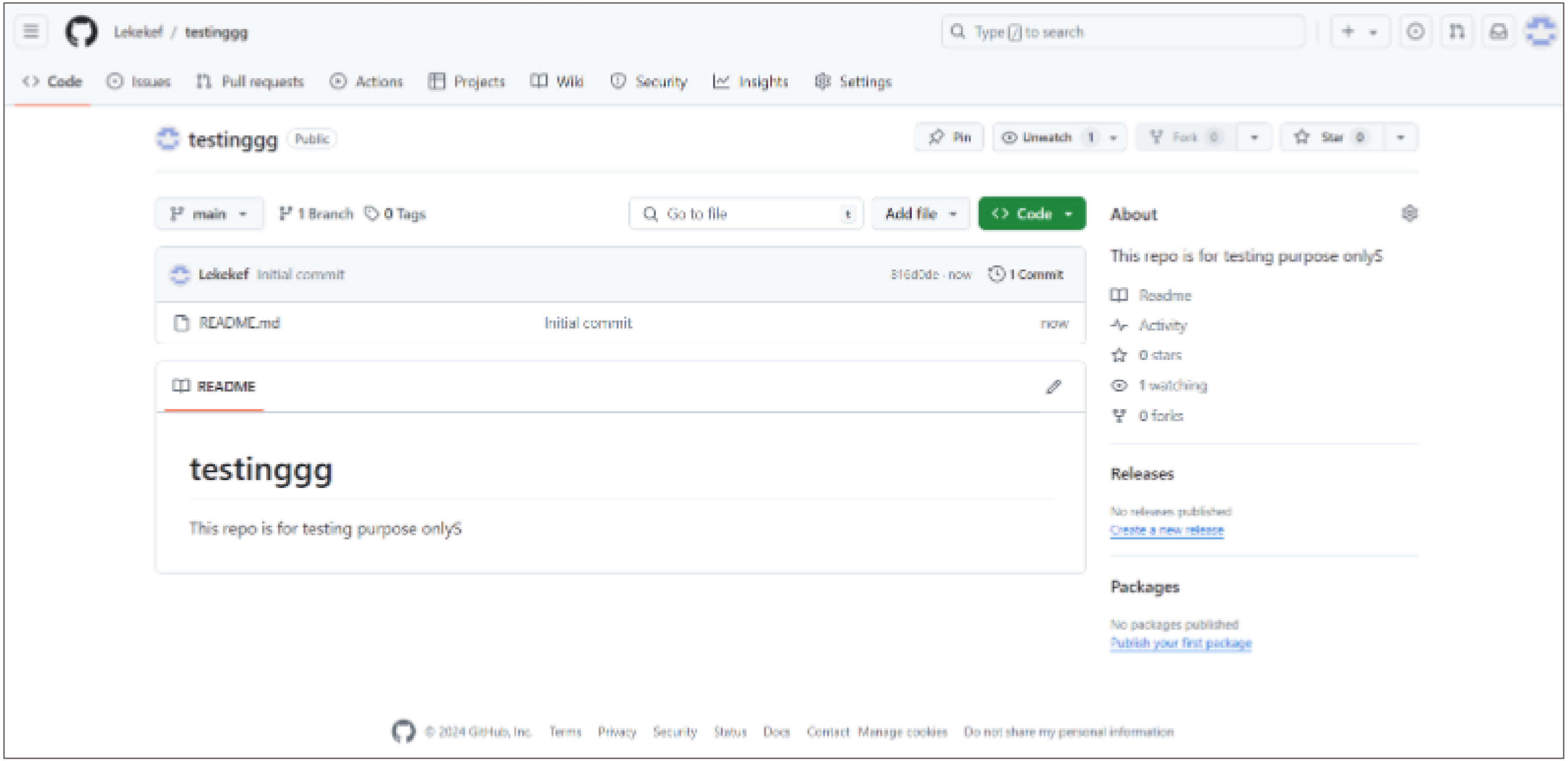
CREATING AND MANAGING REPOSITORIES



CREATING AND MANAGING REPOSITORIES



CREATING AND MANAGING REPOSITORIES



CREATING AND MANAGING REPOSITORIES

2. README File

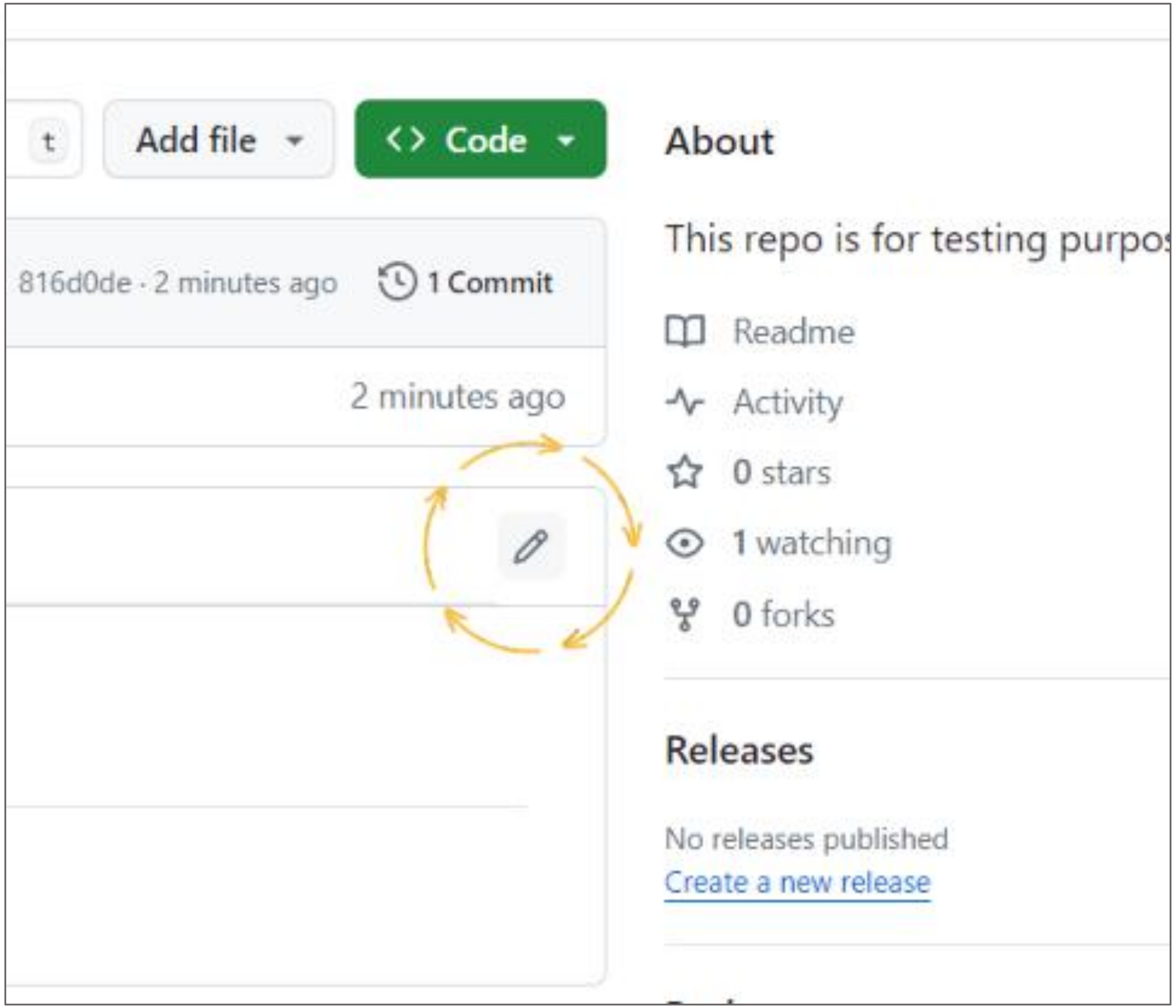
Purpose: A README file provides information about your project, such as what it does, how to use it, and how to contribute.

Steps:

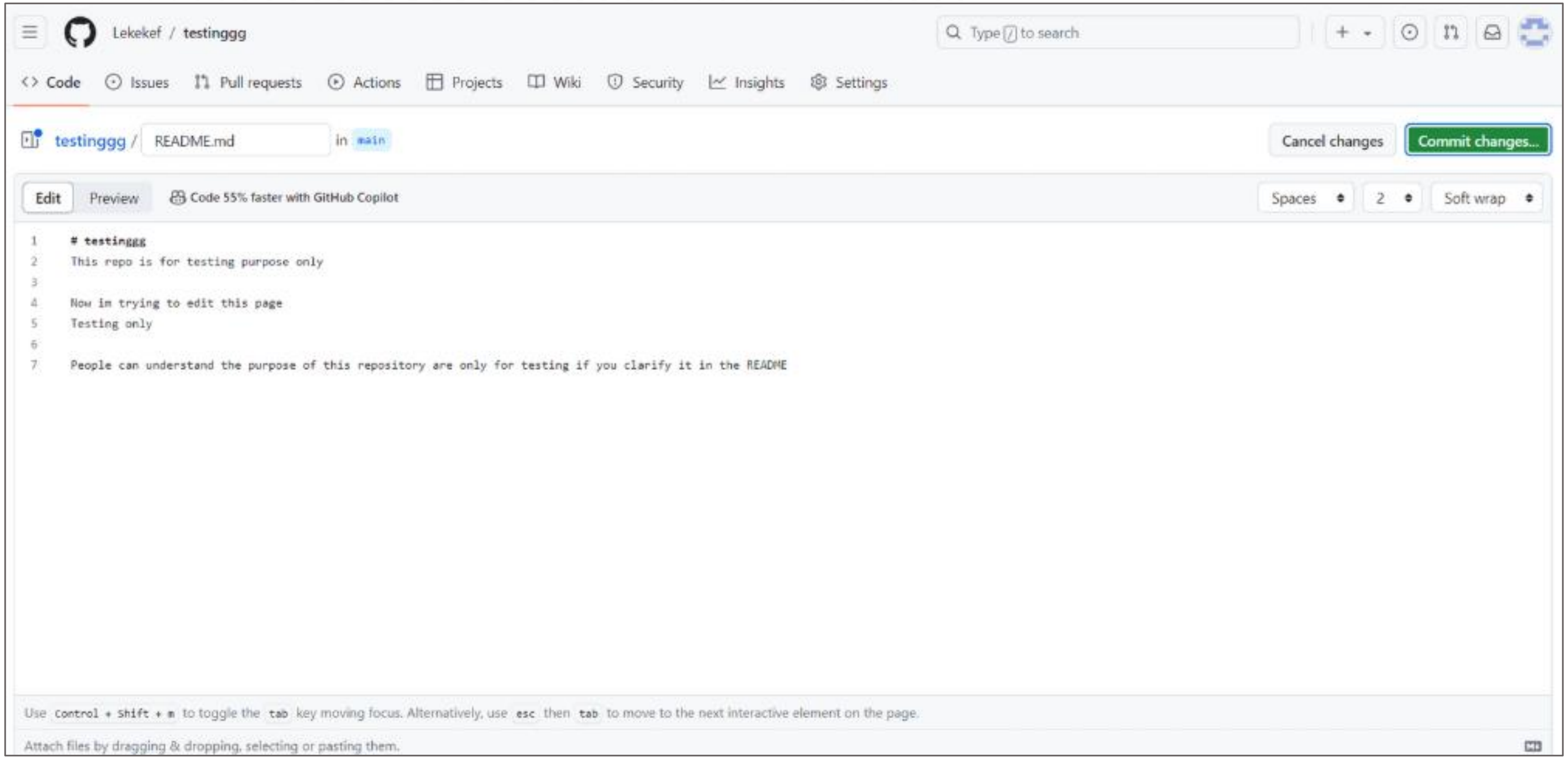
1. Navigate to Repository: Go to your newly created repository.
2. Add README: If you didn't initialize with a README, click "Add a README" on the repository home page.
3. Edit README: Use the GitHub editor to write and format your README in Markdown.
4. Commit Changes: Click "Commit changes" to save your changes.

CREATING AND MANAGING REPOSITORIES

EDIT BUTTON HERE
click this to edit your
README

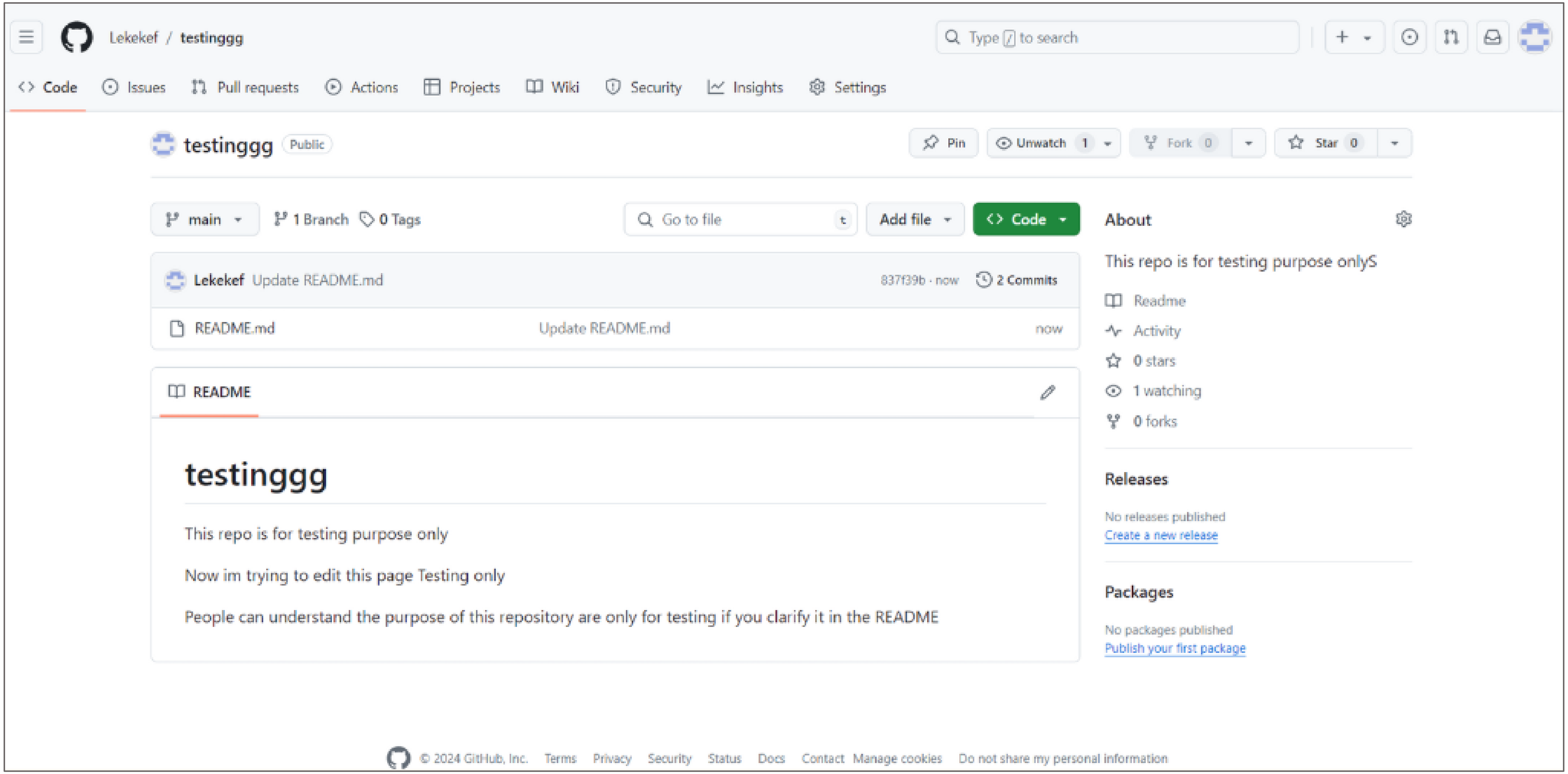


CREATING AND MANAGING REPOSITORIES



You may try and edit your README to give clear descriptions of your projects. Once done, click "Commit Changes" to save your edits.

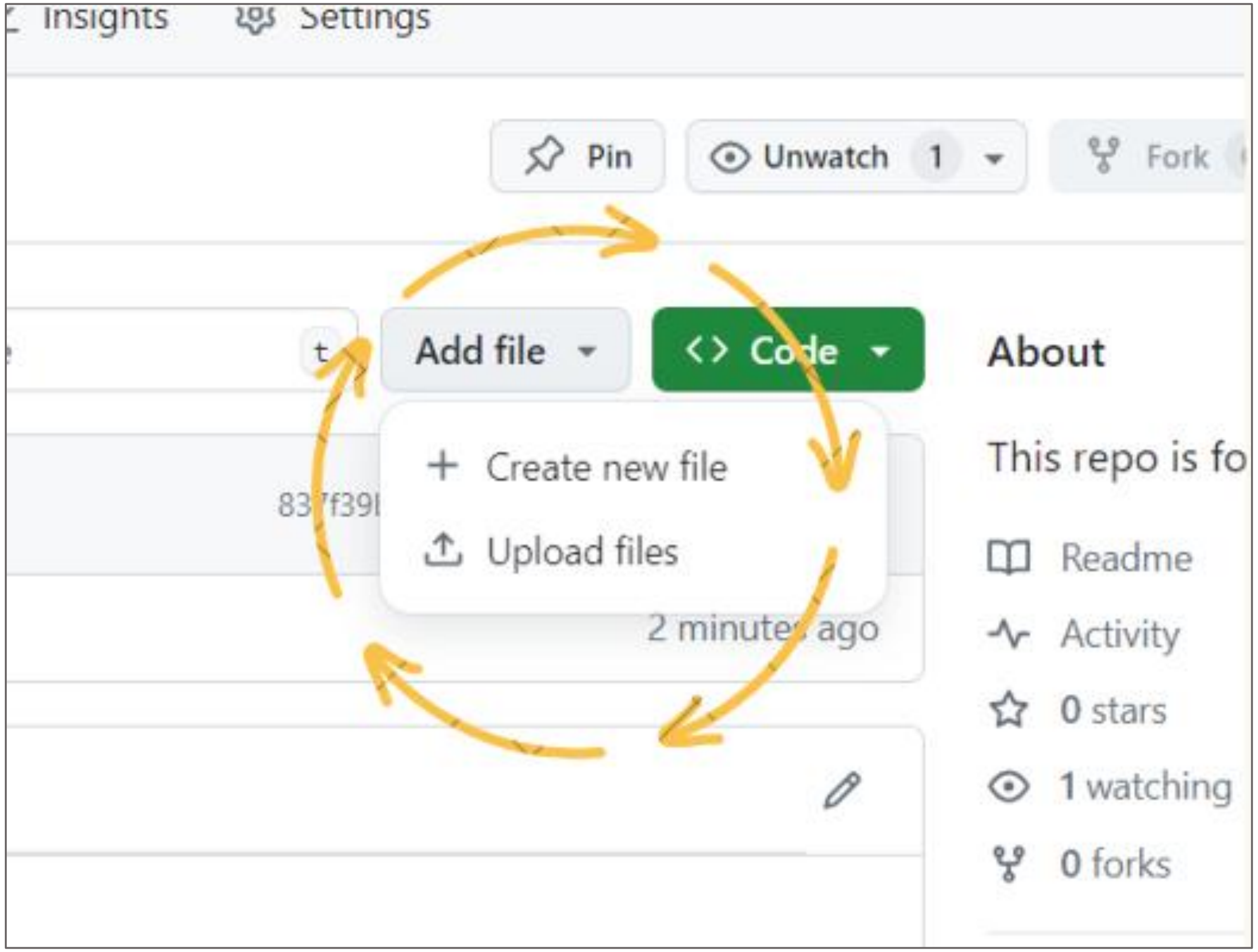
CREATING AND MANAGING REPOSITORIES



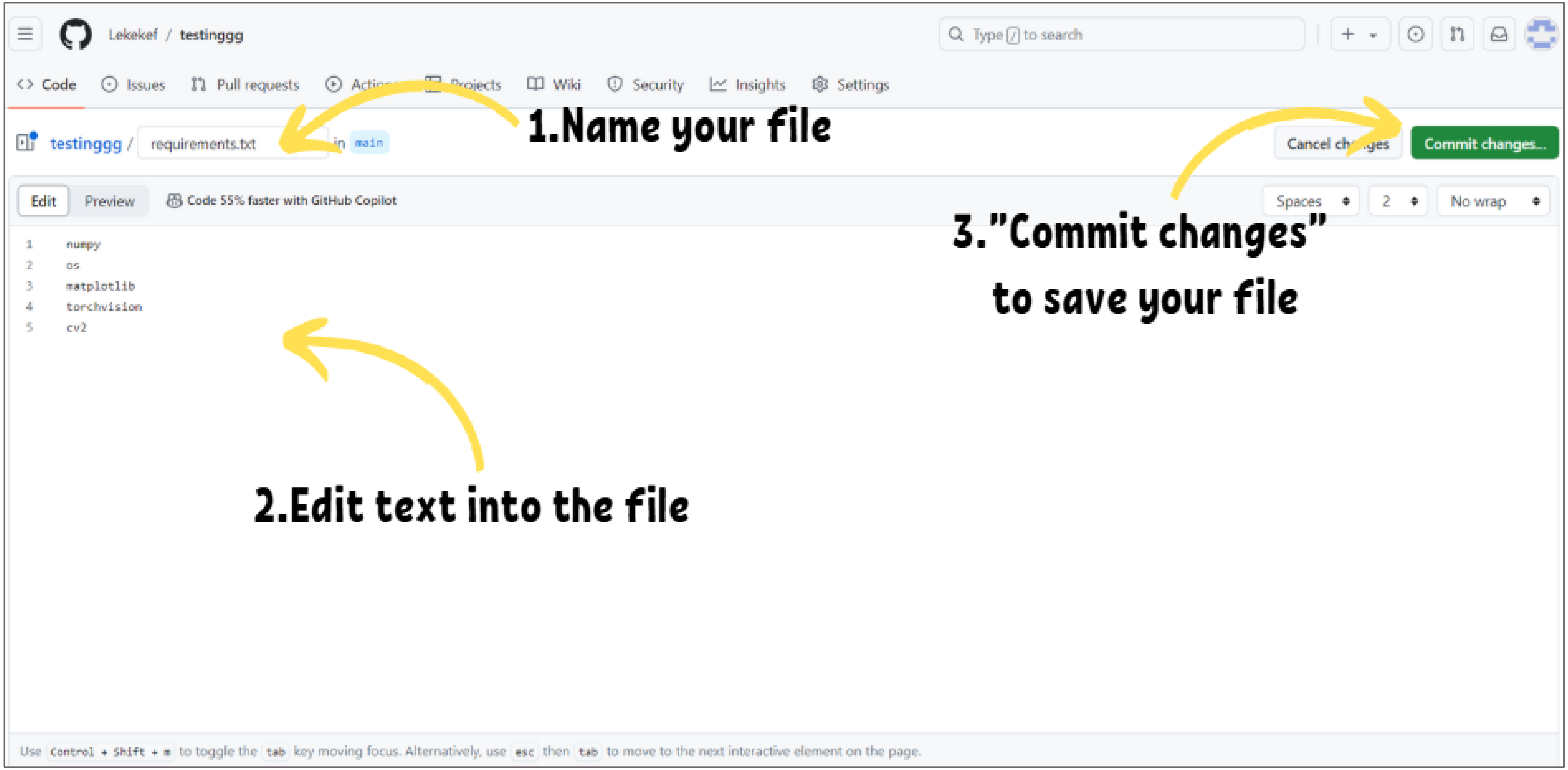
You can view the changes on the main page

CREATING AND MANAGING REPOSITORIES

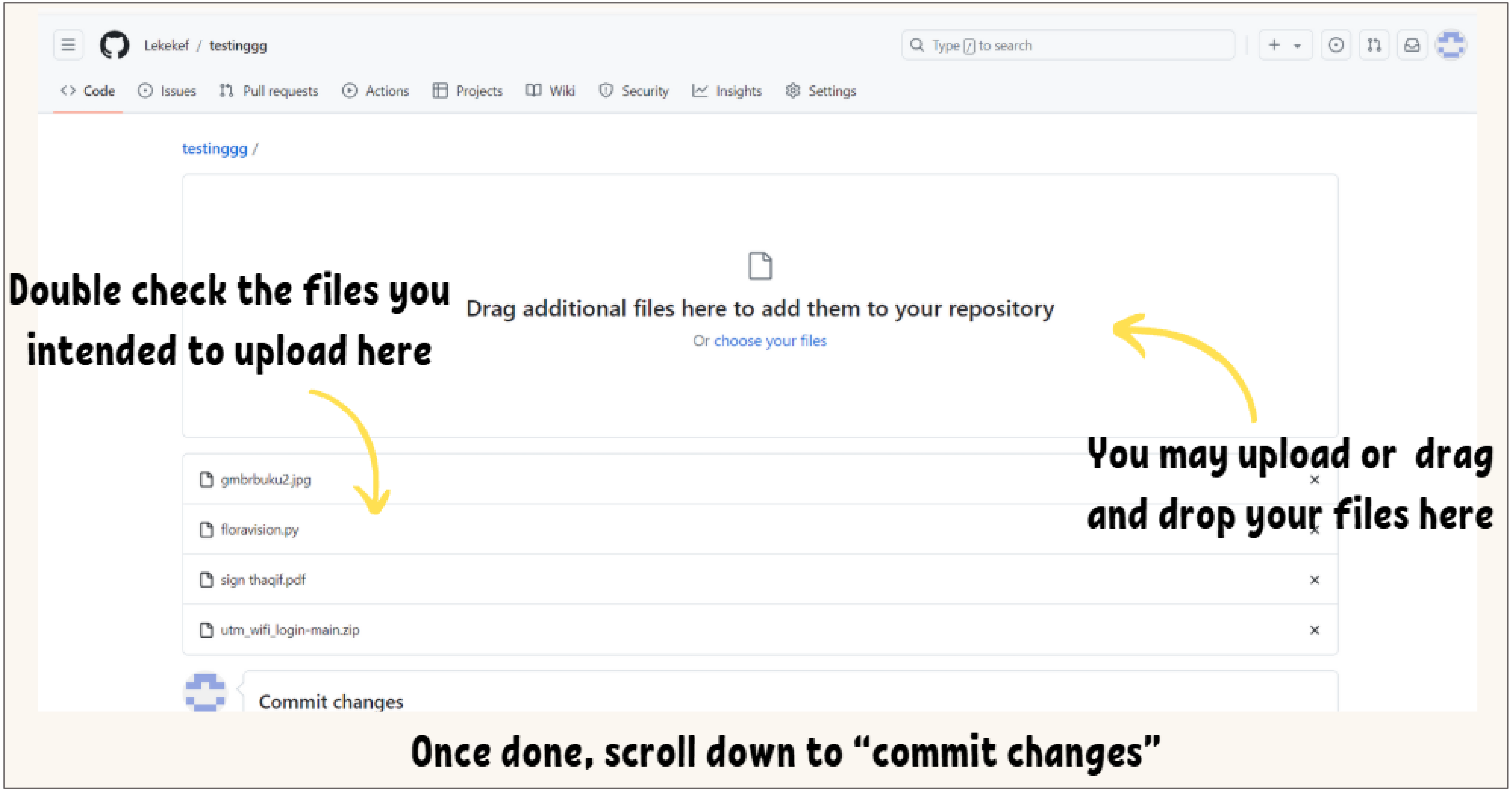
Click here to try and UPLOAD FILES



CREATING AND MANAGING REPOSITORIES



CREATING AND MANAGING REPOSITORIES



UNDERSTANDING BRANCHES

1. Creating Branches

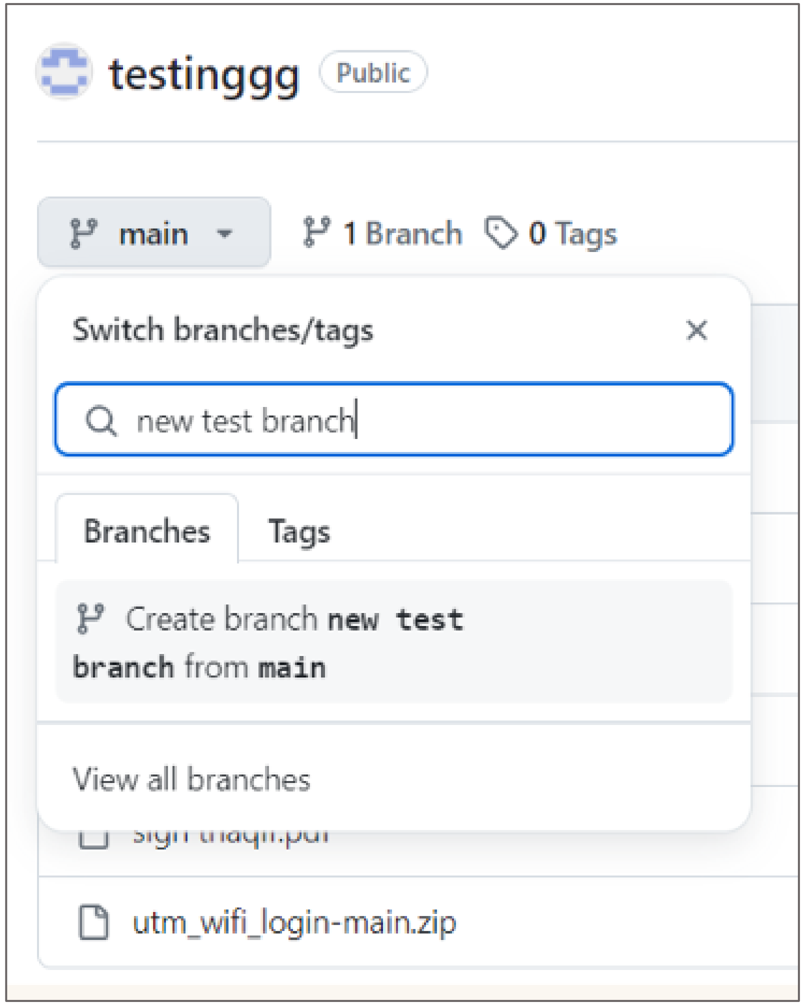
Purpose: Branches allow you to develop features, fix bugs, or safely experiment with new ideas in a contained area.

Steps:

- Navigate to Repository: Go to your repository on GitHub.
- Branch Menu: Click the branch dropdown menu (usually shows "main" or "master").
- Create Branch:

  Type a new branch name into the text box.

  Press "Enter" to create the new branch.



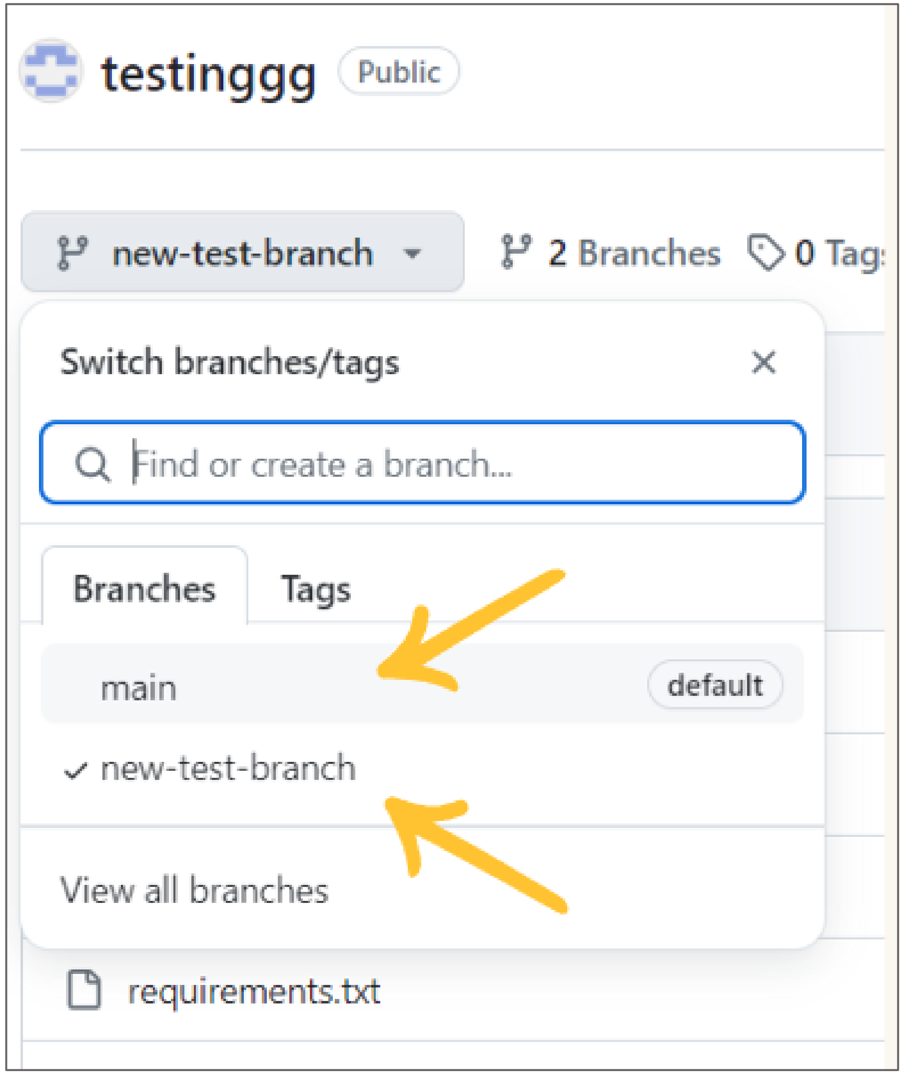
UNDERSTANDING BRANCHES

2. Switching Between Branches

Purpose: Switch between branches to view or work on different versions of your project.

Steps:

- Branch Menu: Click the branch dropdown menu.
- Select Branch: Click on the branch you want to switch to from the list.



UNDERSTANDING BRANCHES

3. Comparing Branches

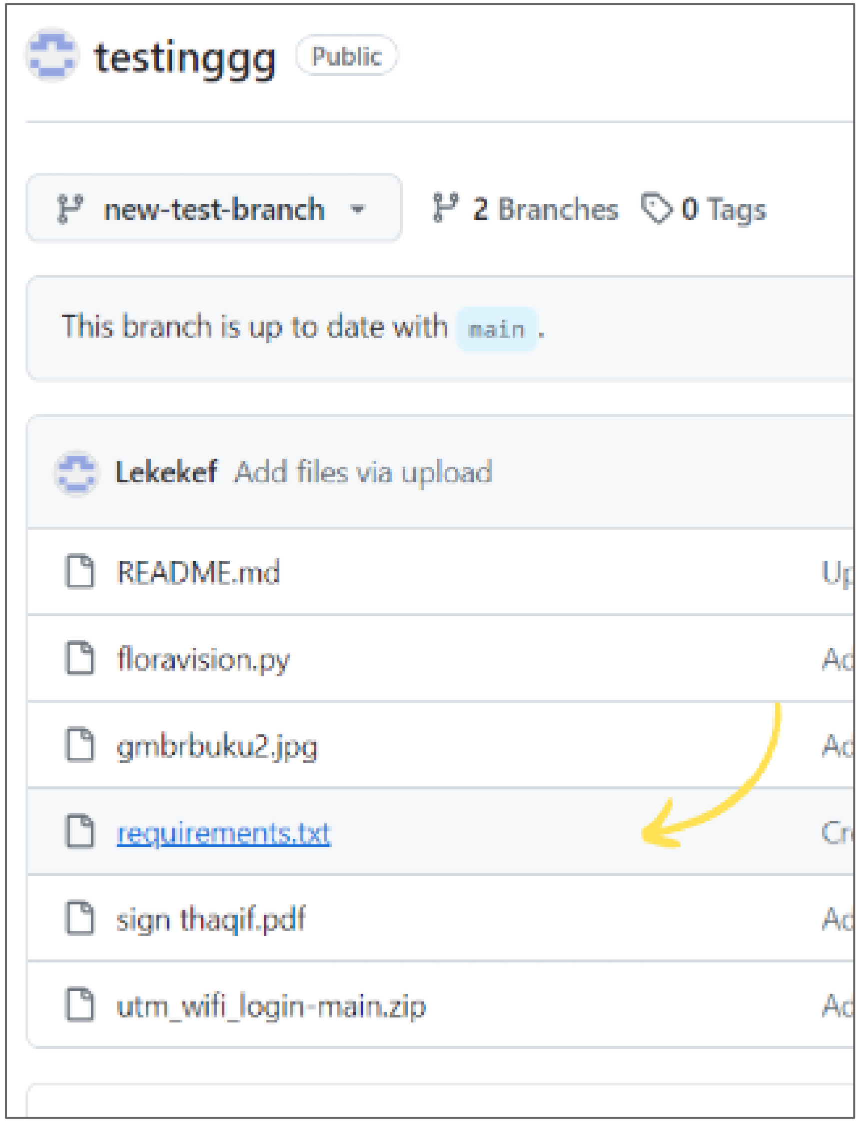
Purpose: Review the differences between branches to understand what changes have been made.

Steps:

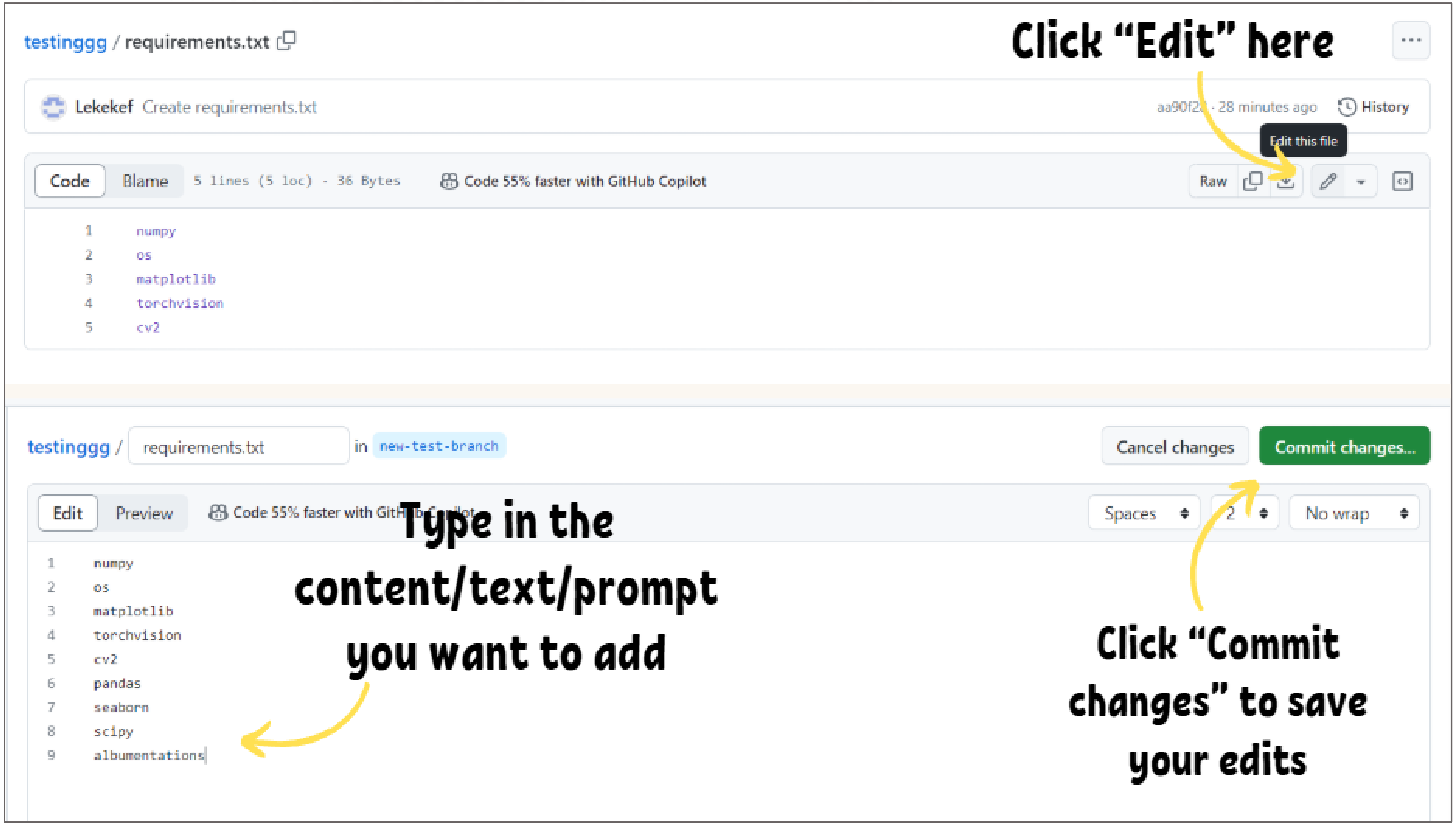
- Navigate to Repository: Go to your repository on GitHub.
- Compare Button: Click the "Compare" button or navigate to the "Pull requests" tab and click "New pull request".
- Select Branches: Choose the base branch and compare branch.
- View Changes: GitHub will show a diff of the changes between the two branches.

UNDERSTANDING BRANCHES

For example here, lets try and edit the content of this file

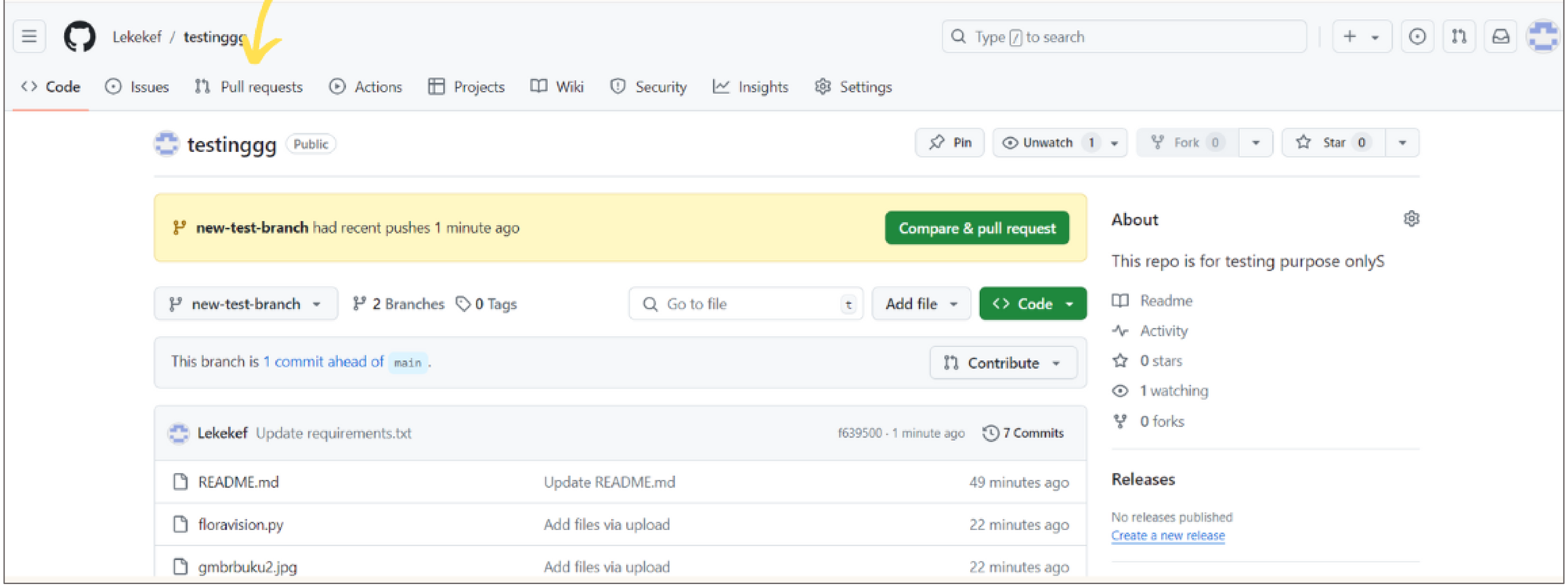


UNDERSTANDING BRANCHES

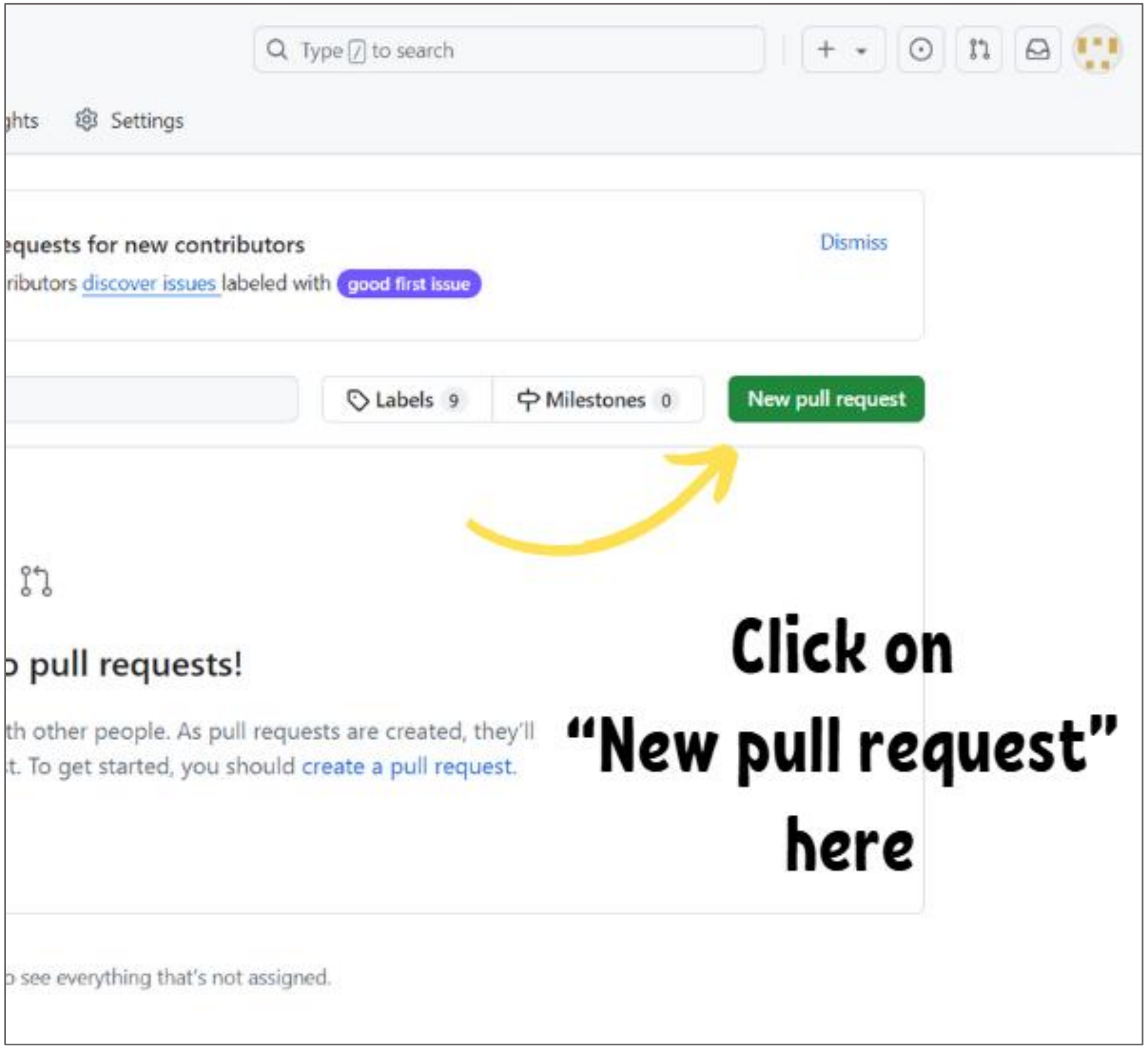


UNDERSTANDING BRANCHES

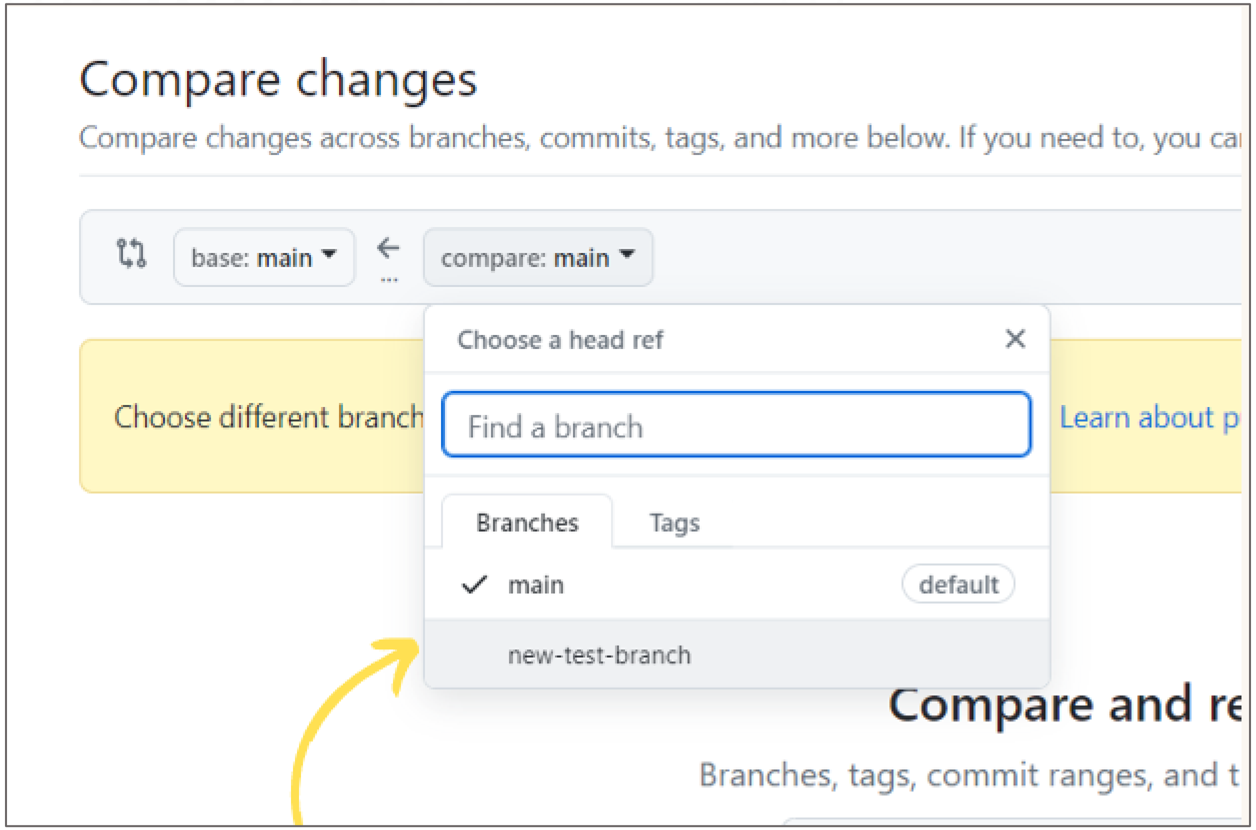
To compare the edited file back with the original file, click on the "Pull Requests" tab



UNDERSTANDING BRANCHES

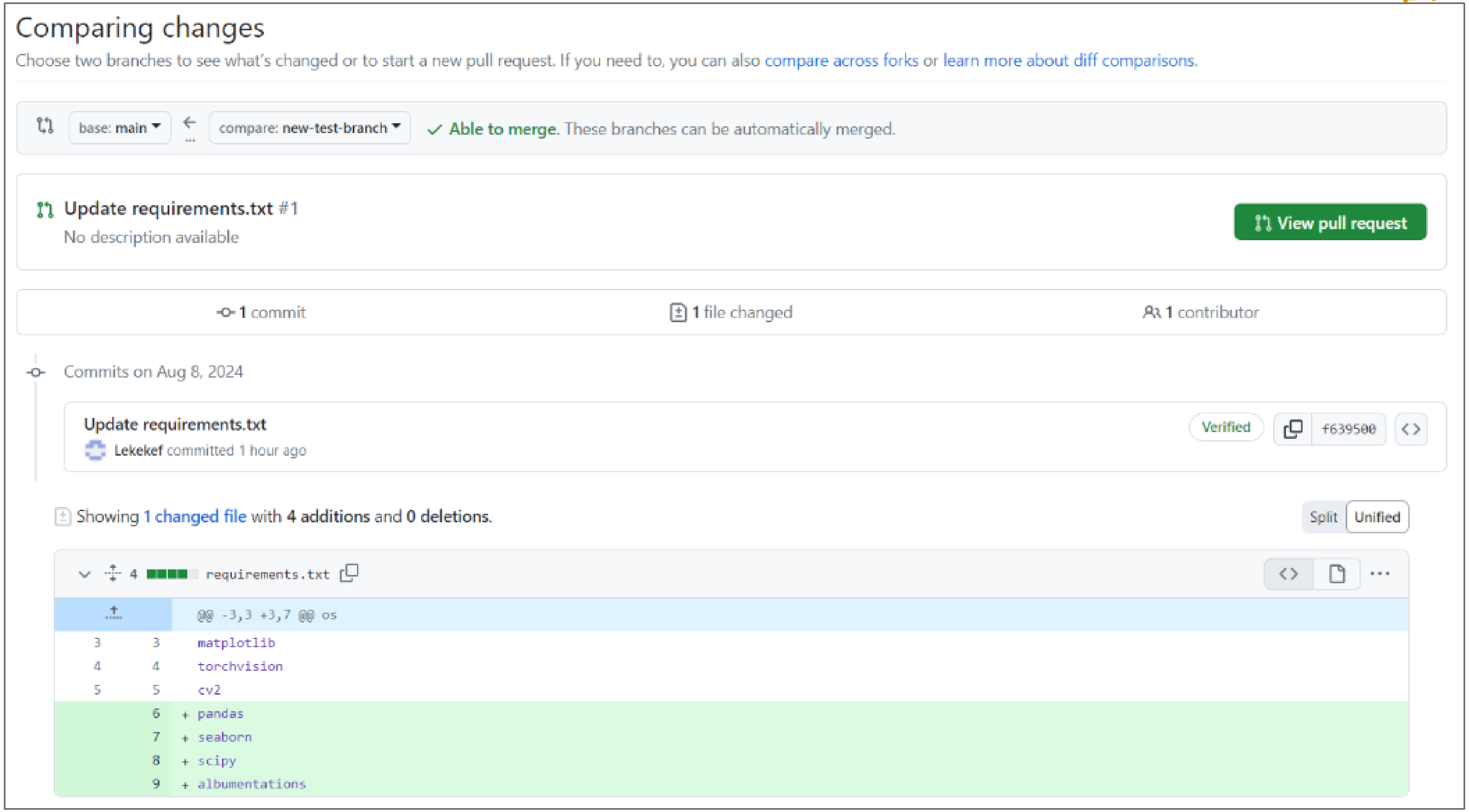


UNDERSTANDING BRANCHES



Next, we are going to compare the 'edited file' in the 'new test branch' with the 'original file' in the 'main branch'. Choose the 'new test branch' here

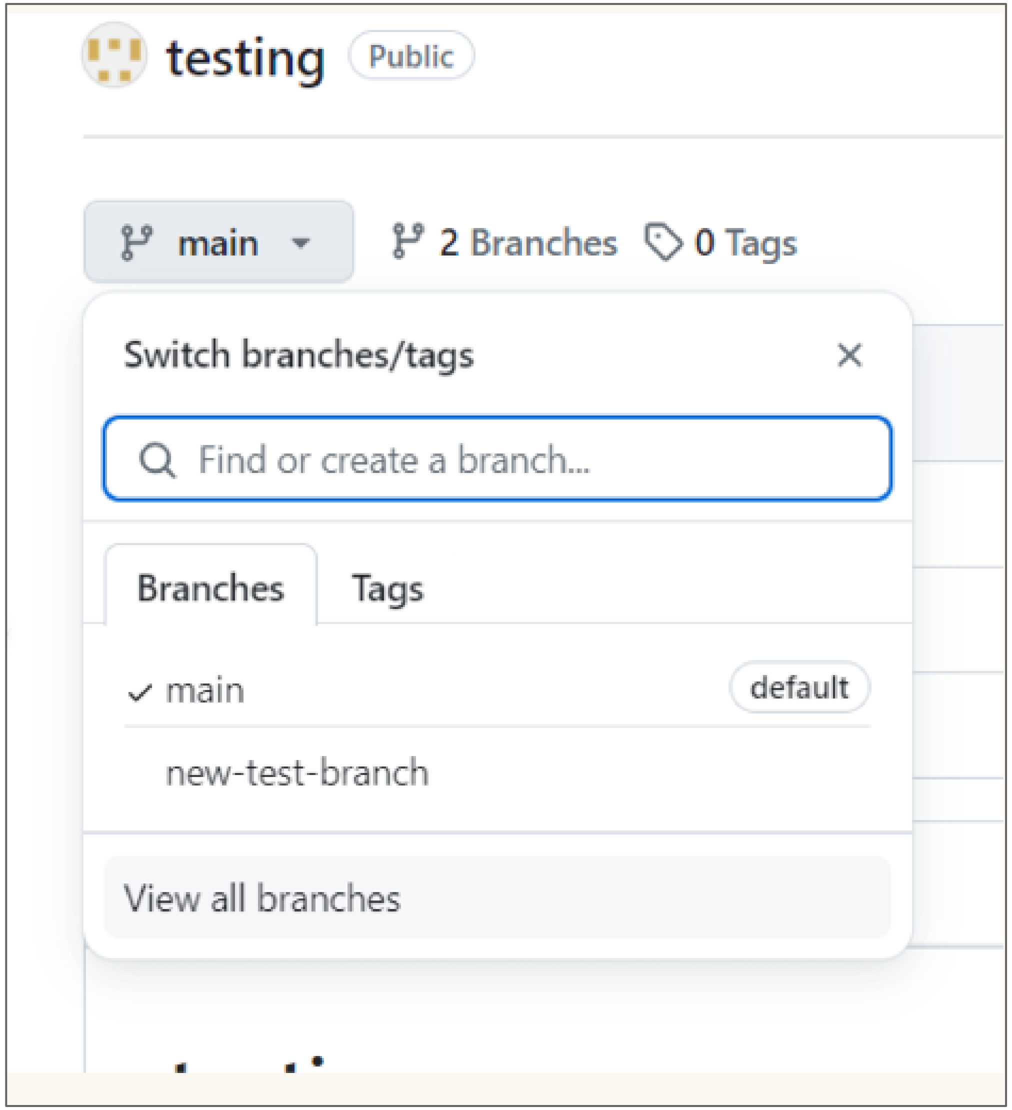
UNDERSTANDING BRANCHES: EXAMPLE



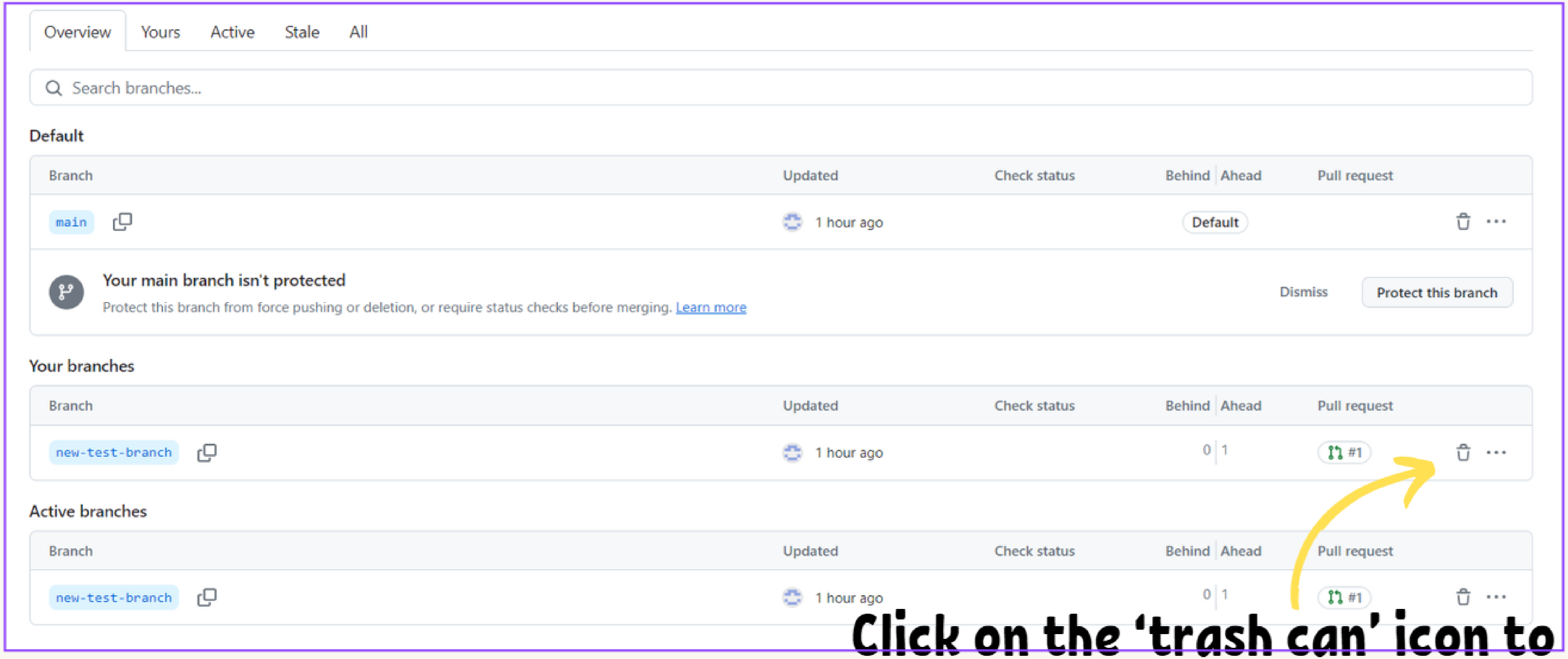
Here, we may view the new input done in the 'new test branch' unified with the original one in the 'main branch'

UNDERSTANDING BRANCHES: DELETE BRANCH

4. Deleting Branches
   - **Purpose**: Clean up unnecessary branches to keep repository organized
   - **Step**:
     - Navigate to Repository: Go to your repository on GitHub.
     - Branches Tab: Click the "Branches" tab.
     - Delete Branch: Click the trash can icon next to the branch you want to delete. Note: You cannot delete the branch you are currently on.

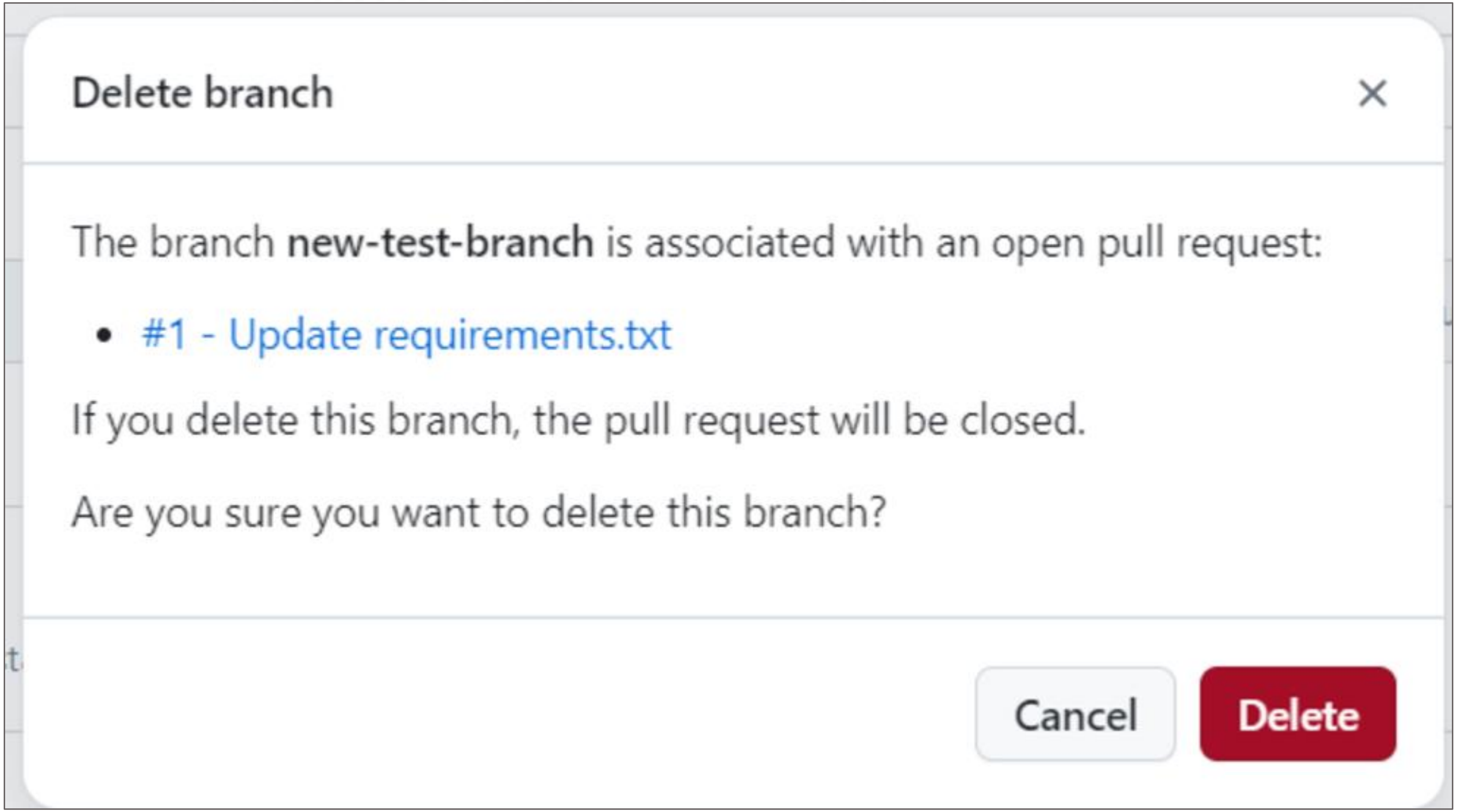


UNDERSTANDING BRANCHES: DELETE BRANCH



Click on the 'trash can' icon to delete your branch

UNDERSTANDING BRANCHES: DELETE BRANCH



ISSUES

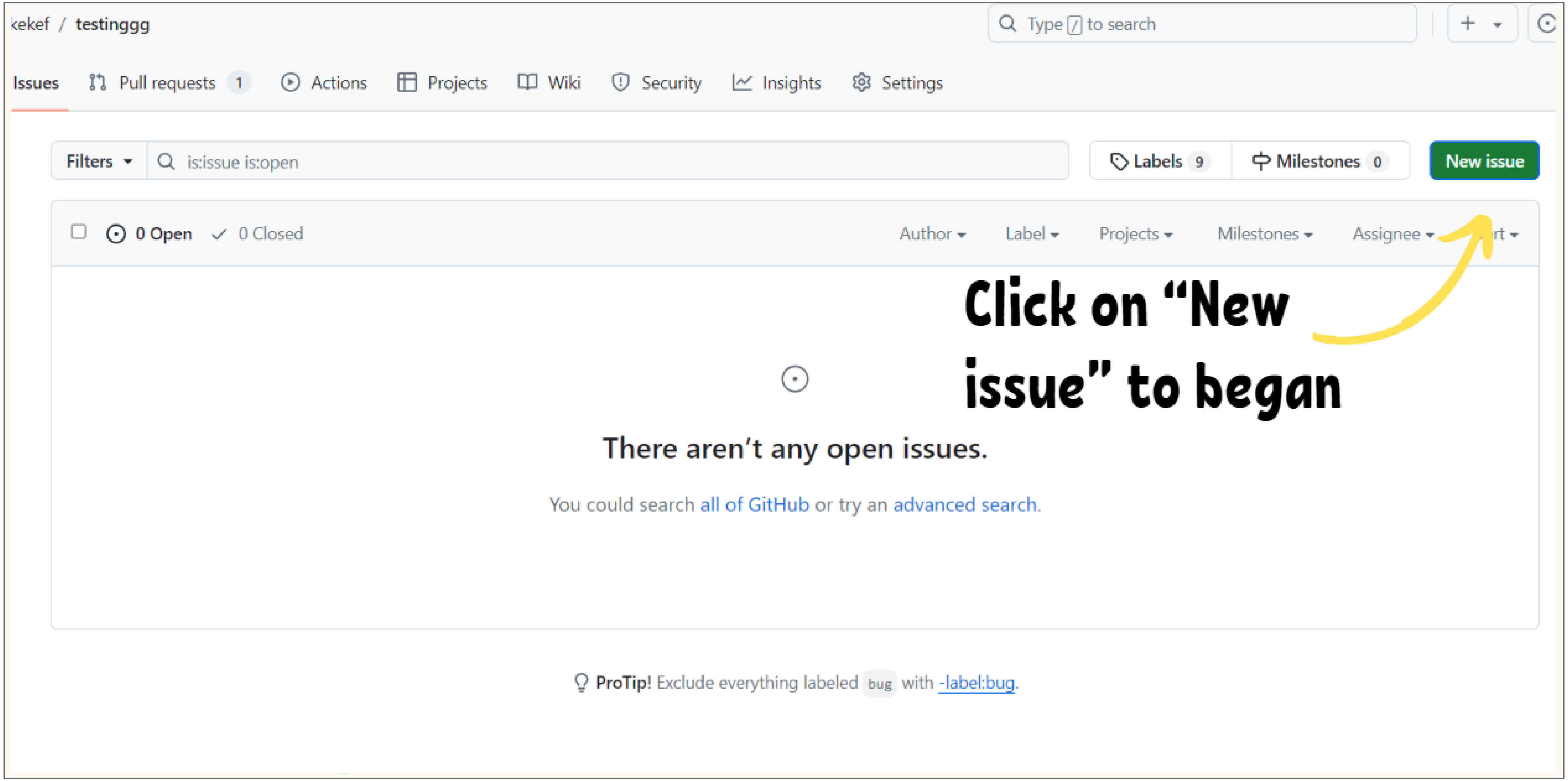
1. Creating and Managing Issues
   - **Purpose:** Issues are used to track tasks, enhancements, and bugs in your project.
   - **Steps:**
     - Navigate to Issues: Go to your repository and click the "Issues" tab.
     - New Issue: Click the "New issue" button.
     - Fill in Details:
       - Title: Provide a concise title for the issue.
       - Description: Describe the issue, including steps to reproduce, expected behavior, and screenshots if applicable.
   - **Submit Issue:** Click “Create" to create the issue.

ISSUES

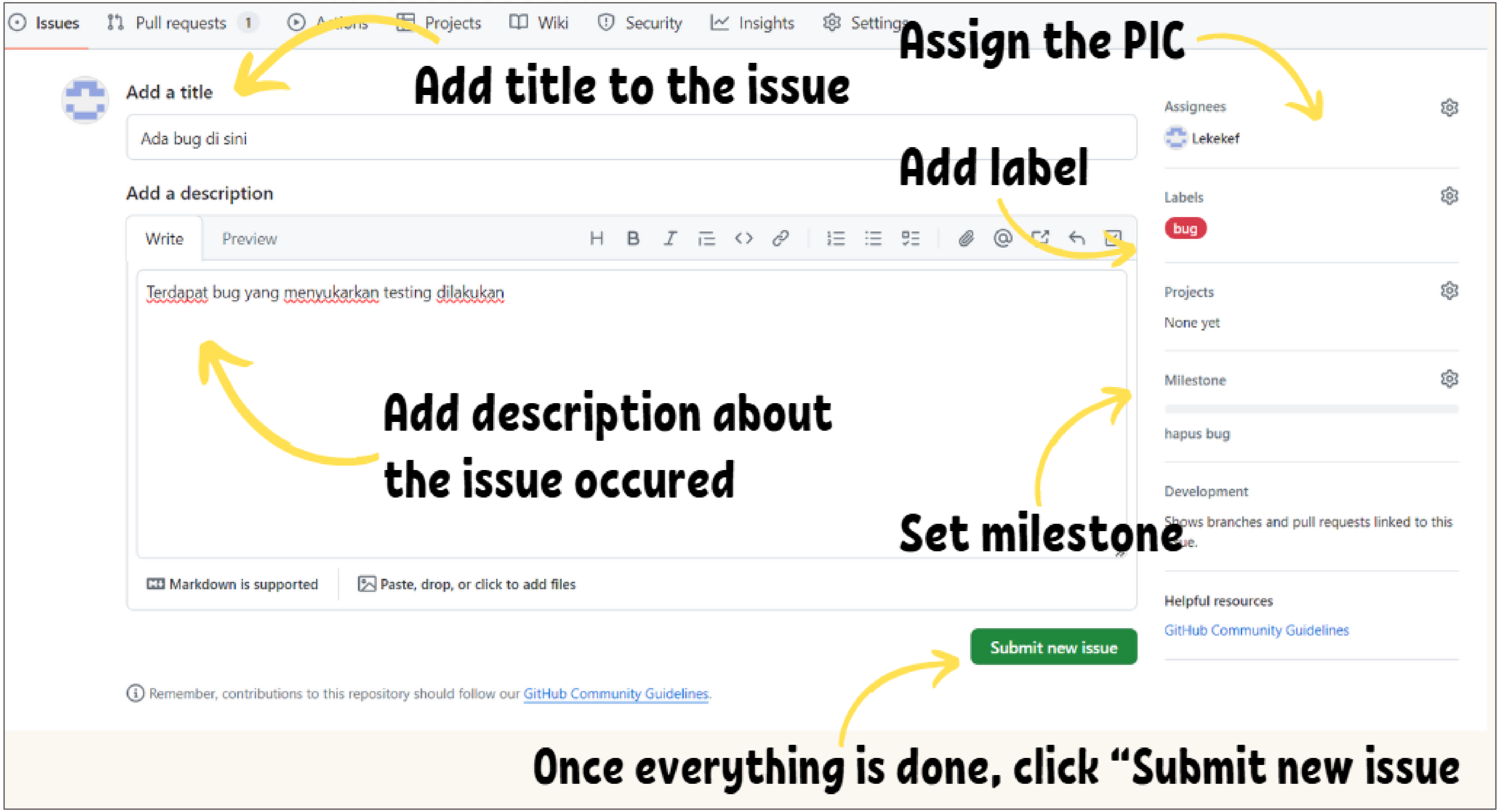
2. Using Labels, Milestones, and Assignees

- **Labels:**
  - Navigate to Issues: Go to the "Issues" tab.
  - Select Issue: Click on an issue to view it.
  - Add Labels: Click the "Labels" button and select or create labels to categorize the issue.
- **Milestones:**
  - Navigate to Issues: Go to the "Milestones" tab within Issues.
  - New Milestone: Click "New milestone", add a title and description, and set a due date if needed.
  - Add Issues to Milestone: Open an issue, click "Milestone", and select the milestone to associate with the issue.
- **Assignees:**
  - Open Issue: Open the issue you want to assign.
  - Assign: Click the "Assignees" button and select the team member to assign the issue to.

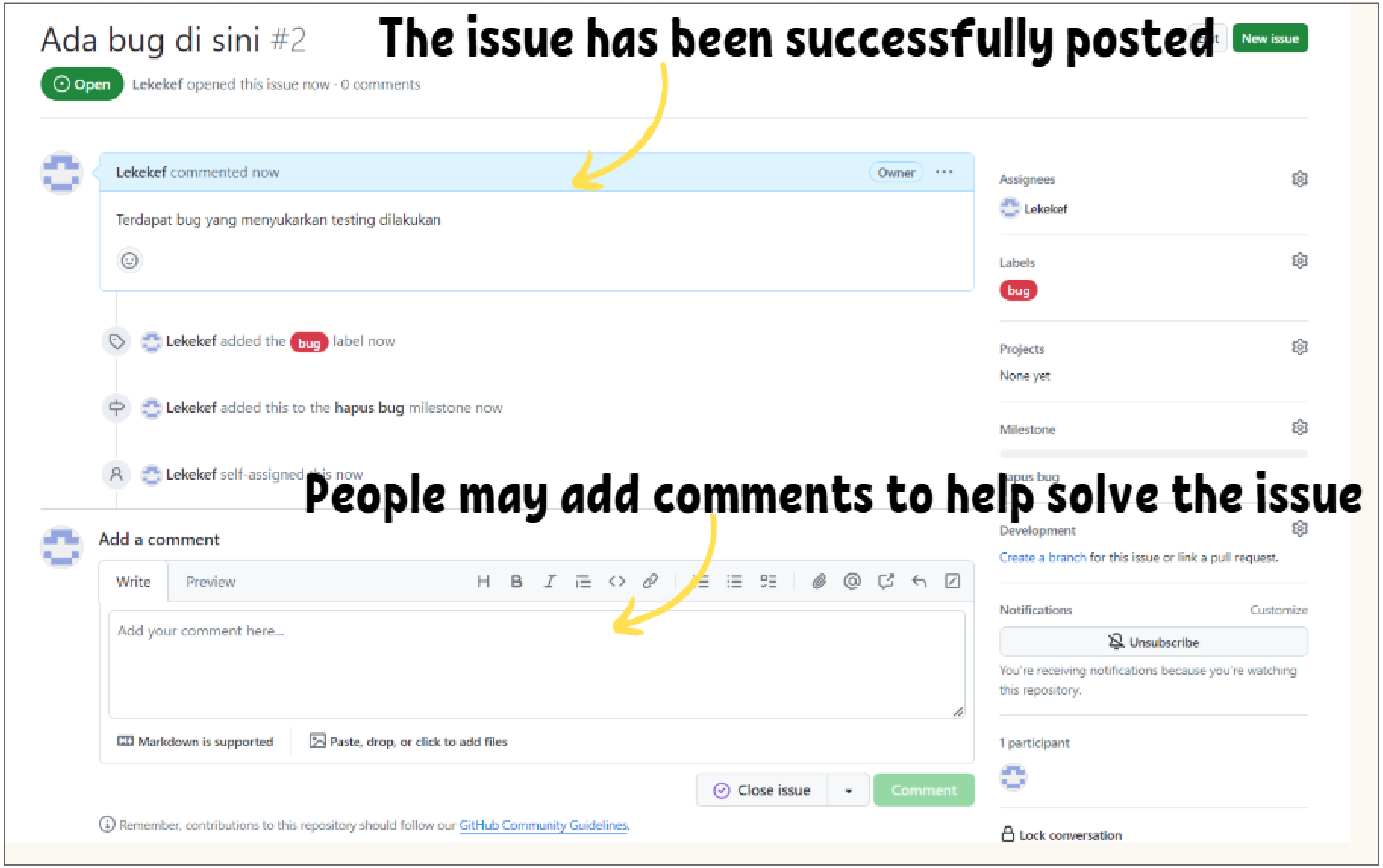
ISSUES: EXAMPLE



ISSUES: EXAMPLE



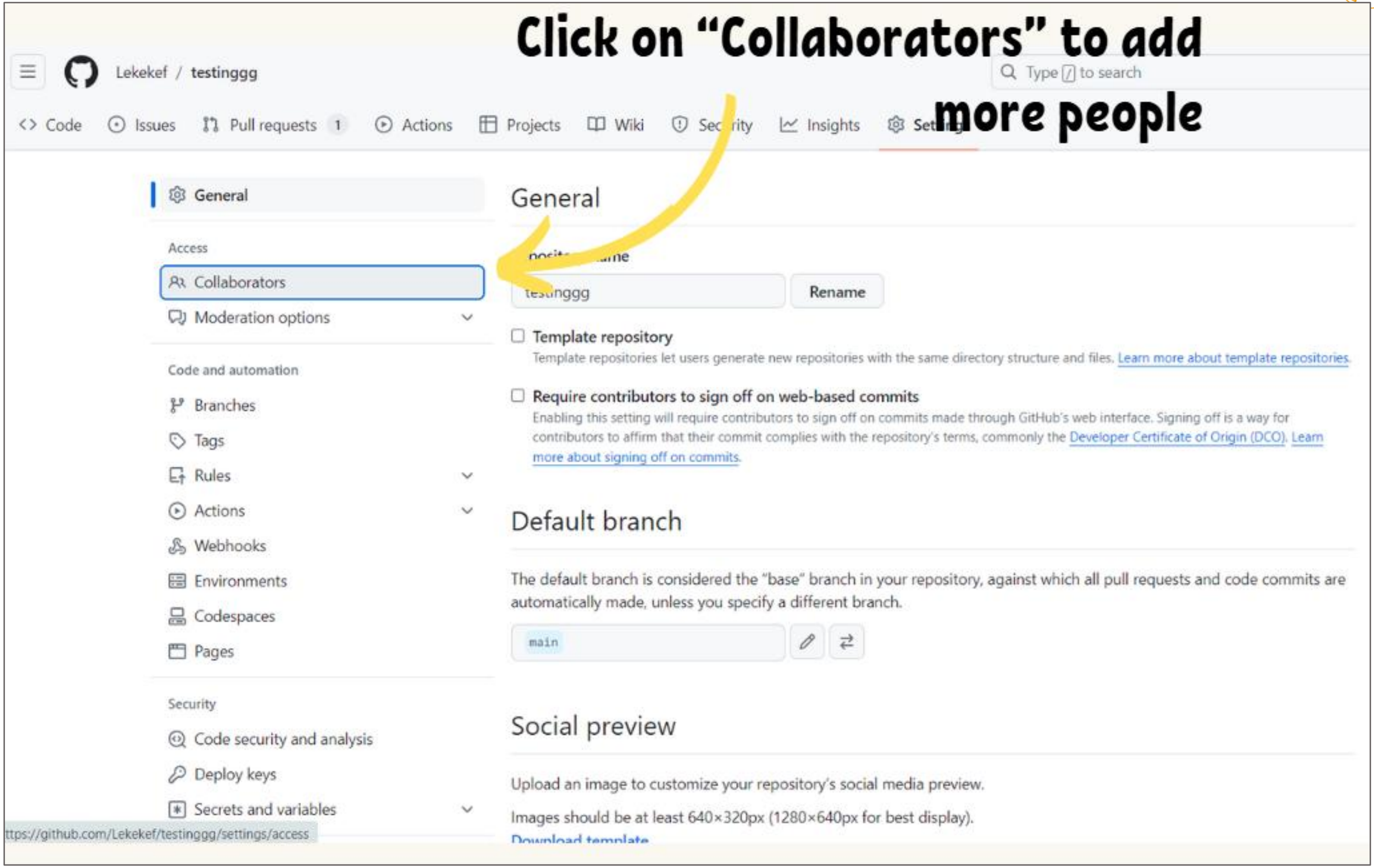
ISSUES: EXAMPLE



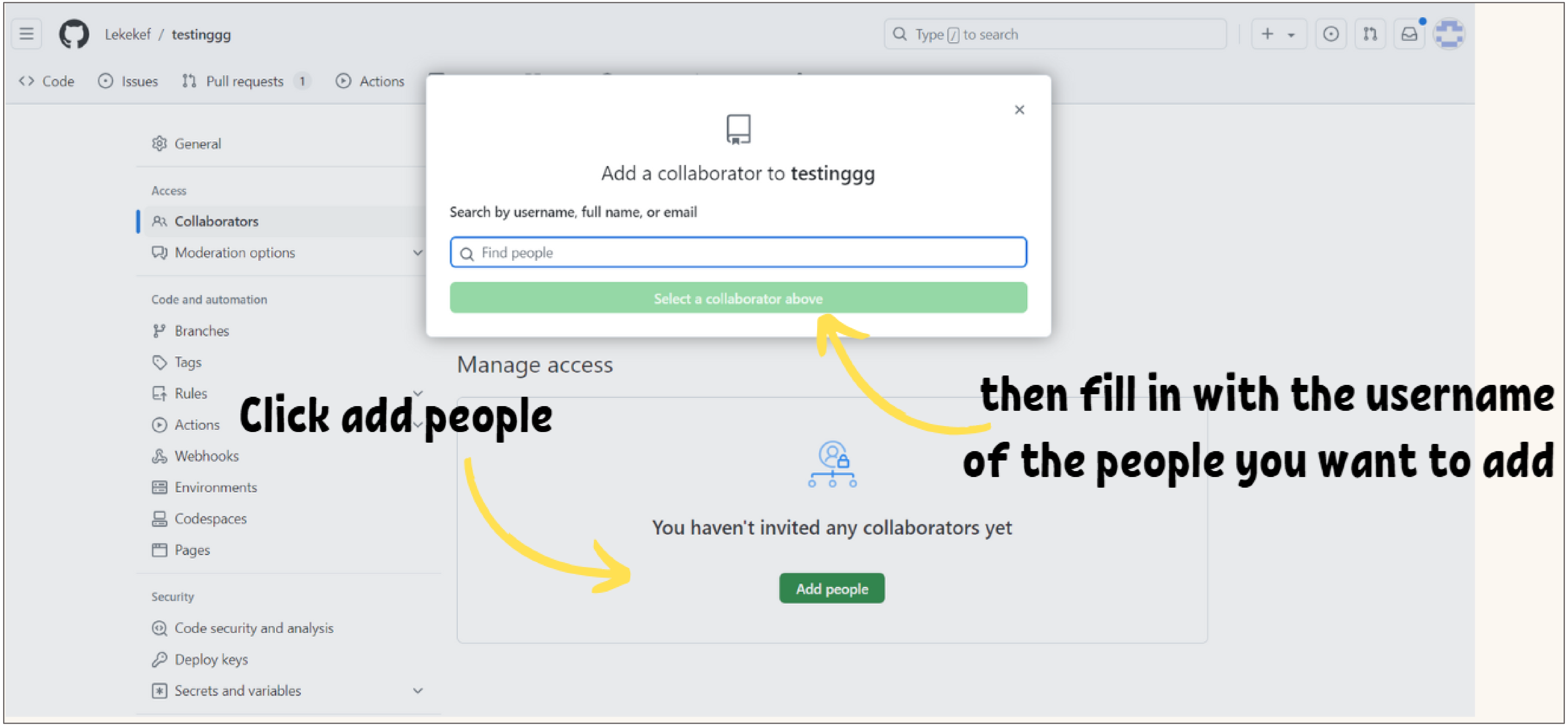
Managing Access Control and Permissions

- **Purpose:** Control who can view and contribute to your repository.
- **Steps:**
    - Navigate to Settings: Go to your repository and click the "Settings" tab.
    - Manage Access: Click "Manage access".
    - Add Collaborators: Click "Invite a collaborator" to add users and set their permissions (Read, Write, Admin).

SECURITY ON GitHub: EXAMPLE

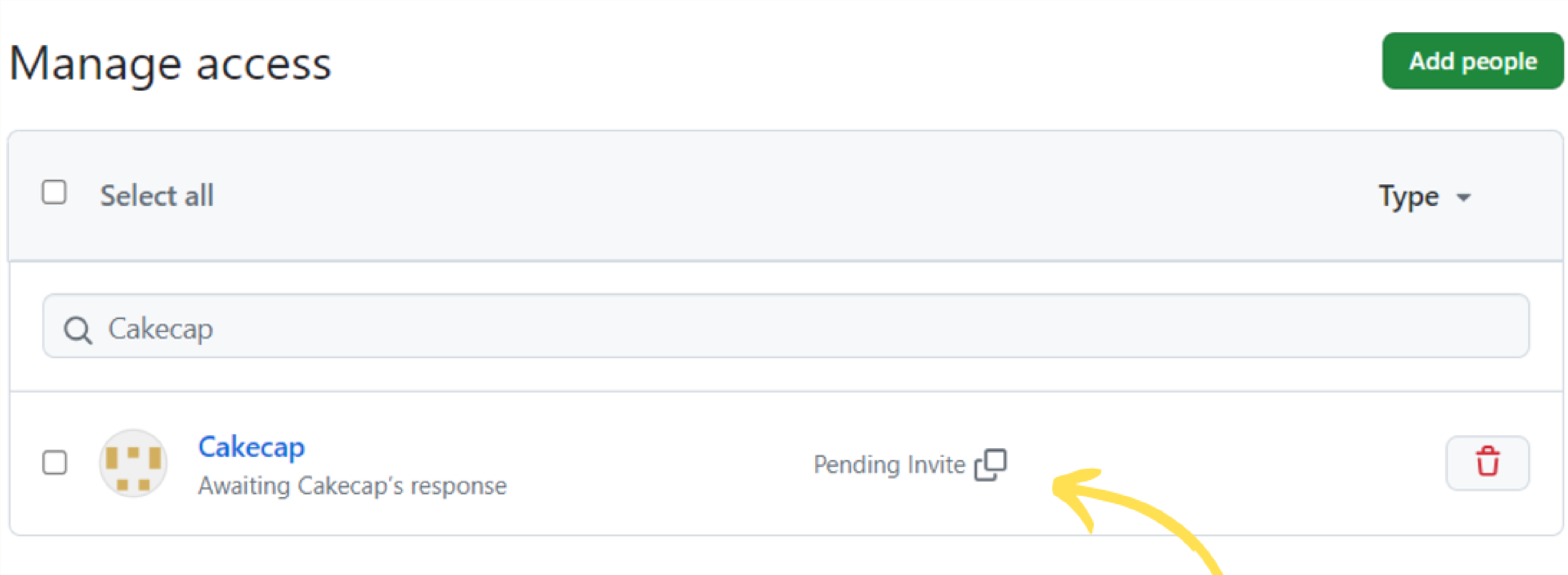


SECURITY ON GitHub: EXAMPLE



SECURITY ON GitHub: EXAMPLE

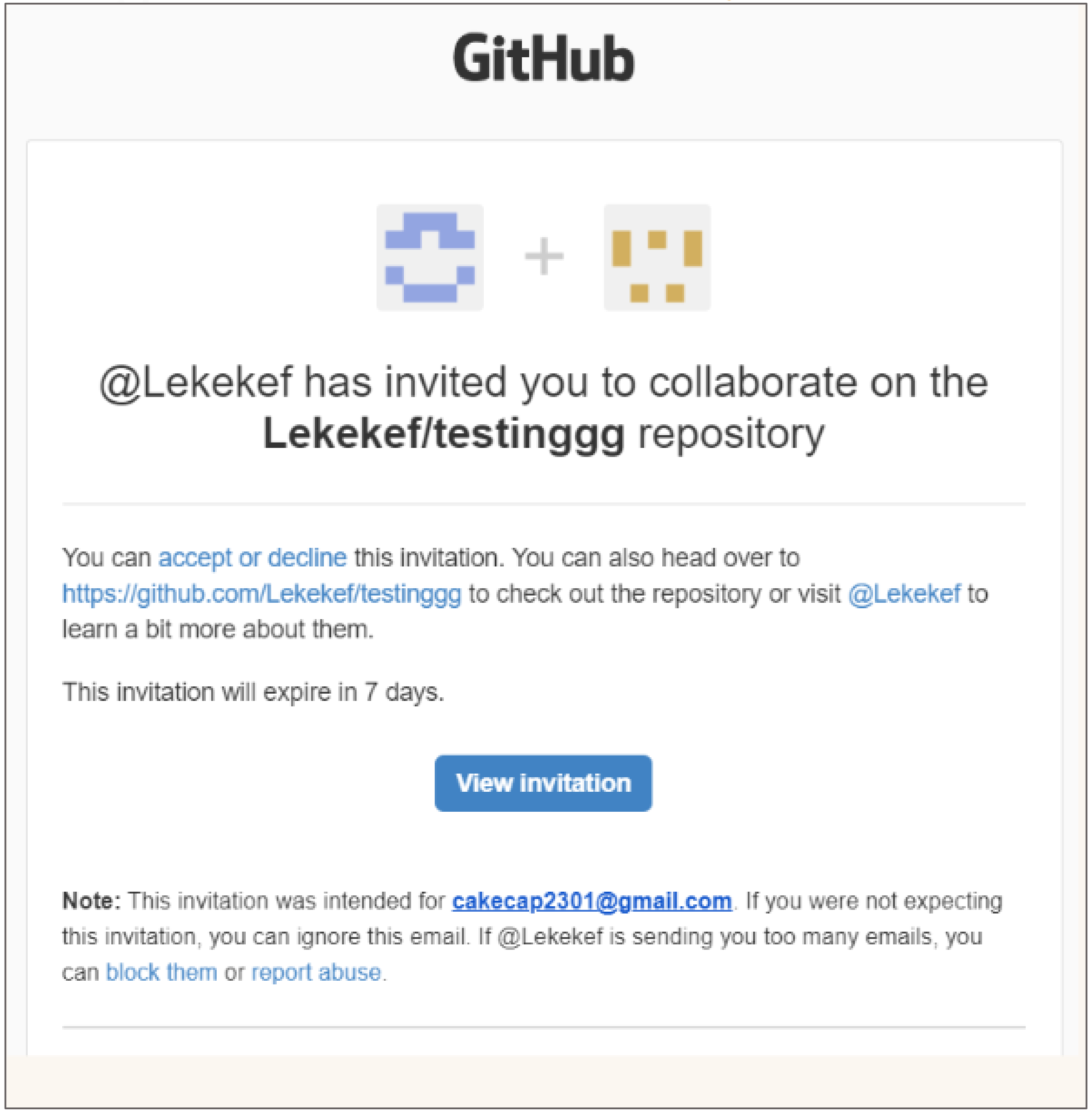
Once done, we may view the list of people we want to add.



But the invite is actually still pending. So, we need to wait until they accept it via email.

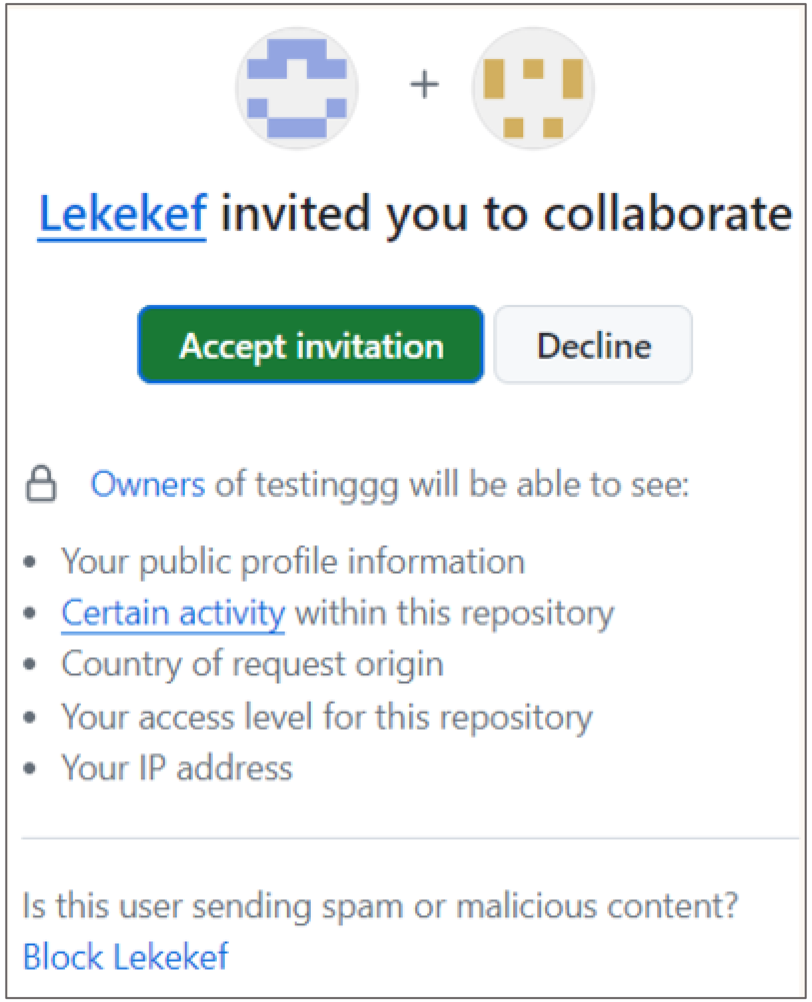
SECURITY ON GitHub: EXAMPLE

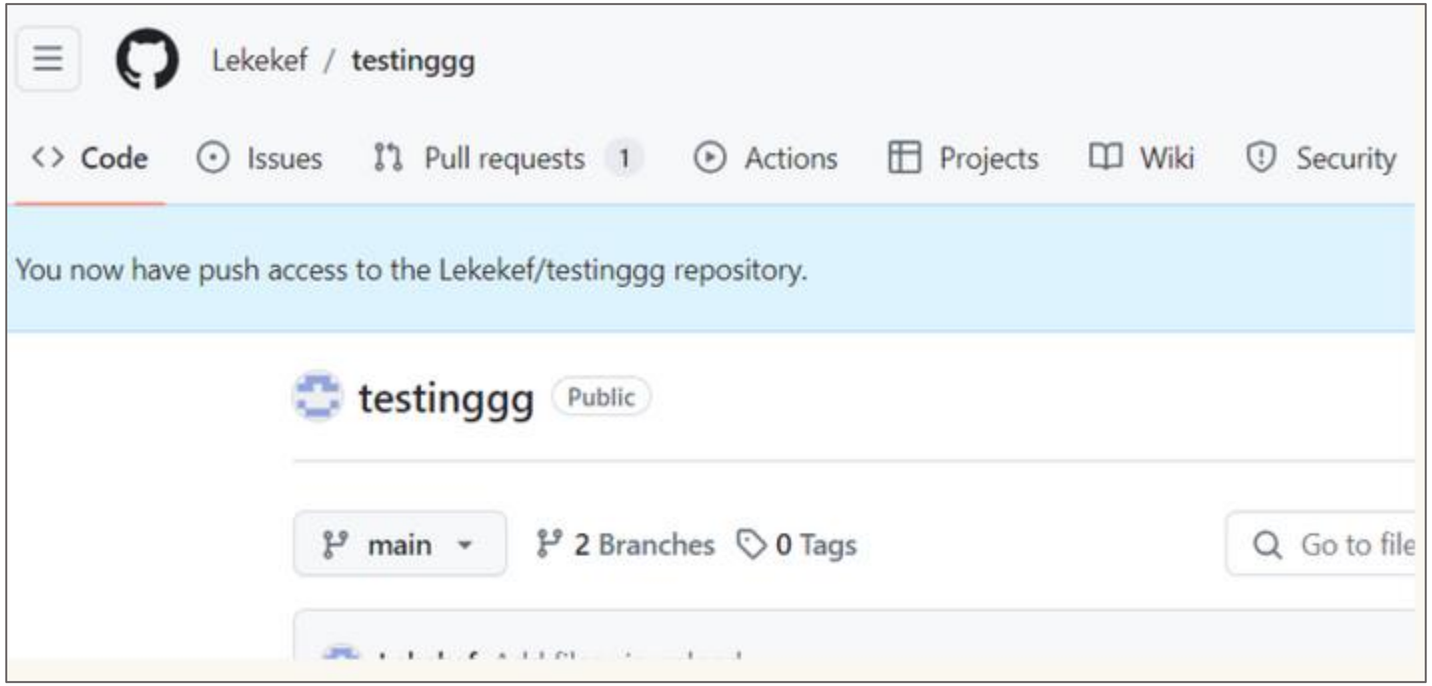
The invited people should get the invitation via email. Click on the "View invitation" to proceed.



SECURITY ON GitHub: EXAMPLE

Once the invitation is accepted, we may see a notification pop-up indicating the other user has been successfully added





SETTING UP GitHub IN JETSON

Installing Git

- Install Git on your Jetson Orin:

sudo apt update

sudo apt install git

Configuring Git

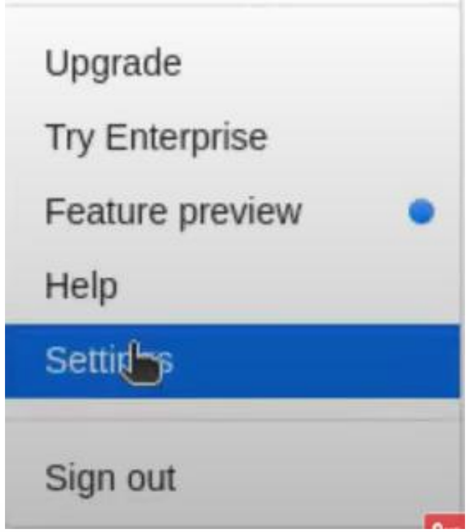
- Set up your Git username and email:

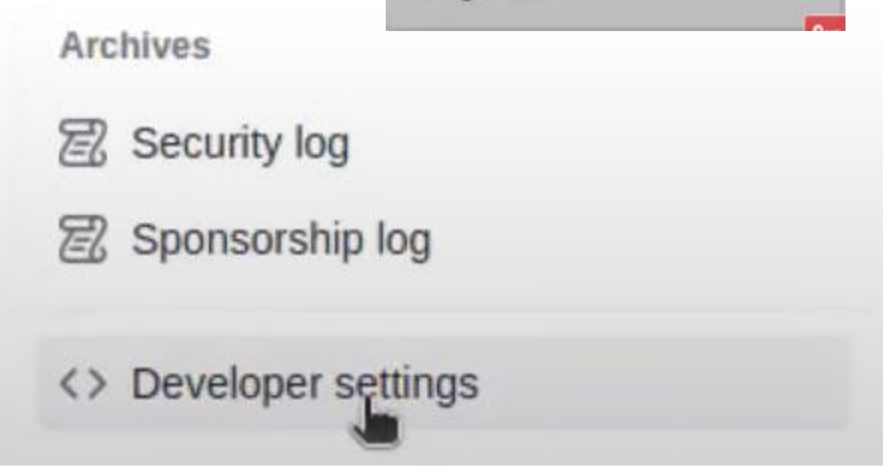
git config --global user.name "Your Name“

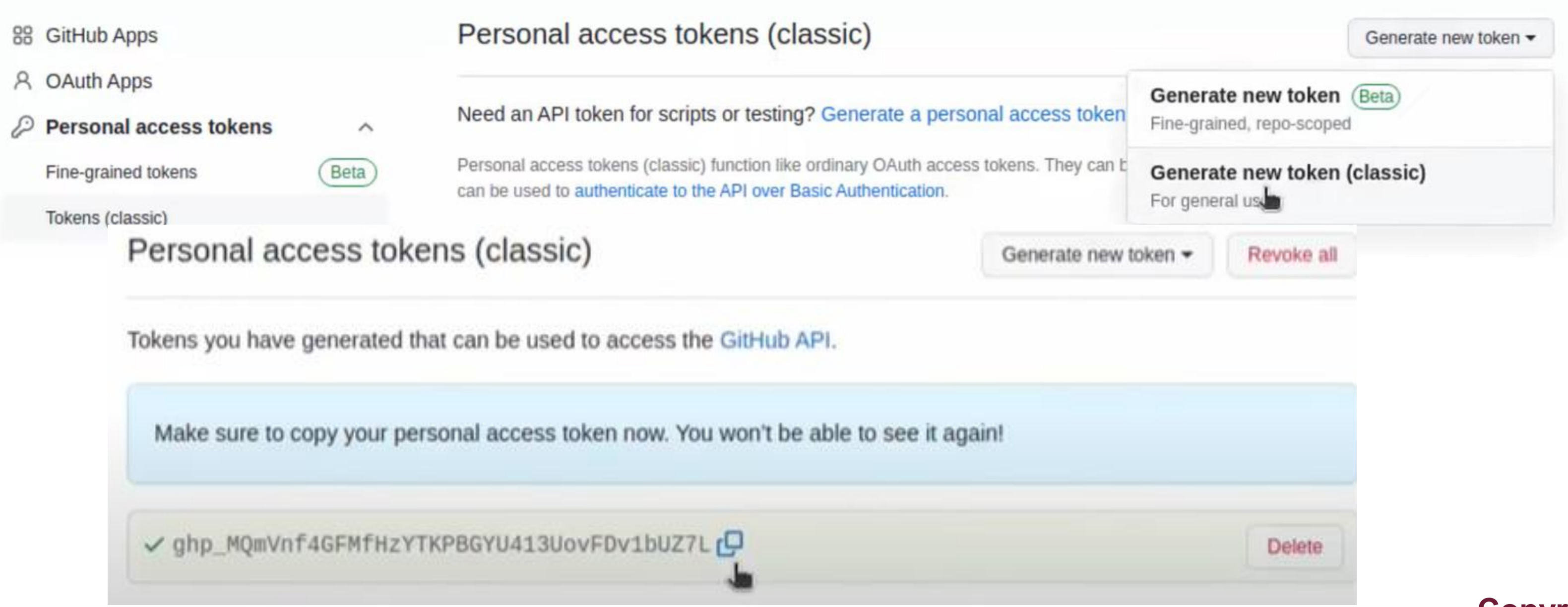
git config --global user.email "your.email@example.com"

ACCESSING GIT

1. From a GIT client application, you authenticate to the GitHub server by using your **authenticate username** and a **Personal Access Token (PAT)**.
2. Select your profile icon in the top corner, select setting.
3. Then in the left panel select “Developer setting“
4. Click “Personal access tokens” , then generate new token (classic)
5. Fill in the note and aelect all the scope in generation and generate the token.
6. Copy the token (remember this only will show once)







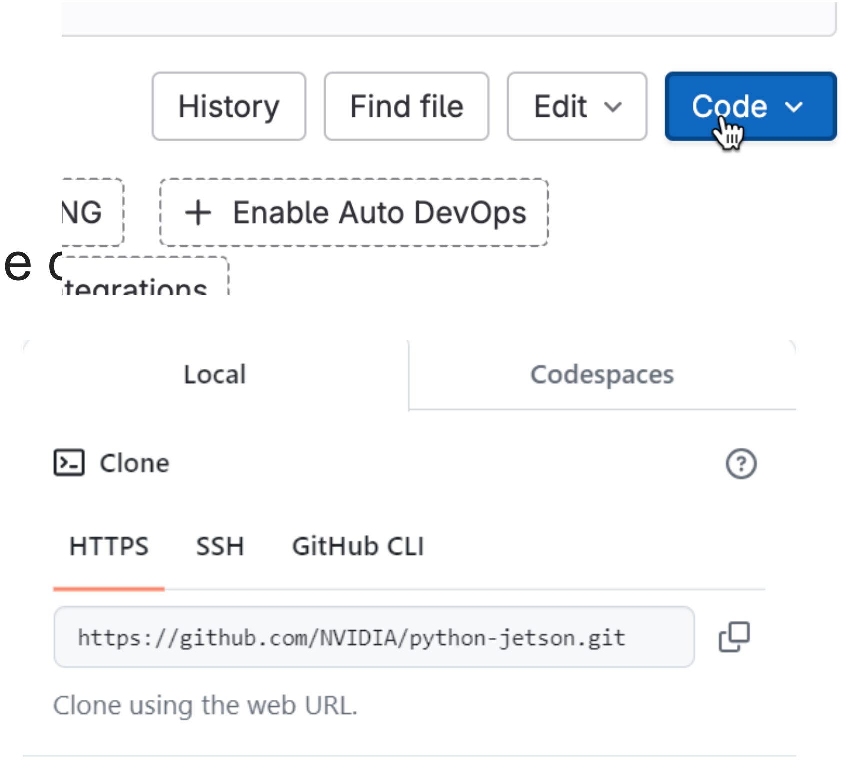
JETSON TO AND FROM GitHub

- On the GitHub select your project, select Code on the right and then select the clone URL
- Use the 'git clone' command followed by the repository URL.

git clone https://GitHub.com/username/repository.git

- After the cloning process completes, you can navigate into the cloned repository's directory:

Cd repository



- Adding Files to Git:

echo "print('Hello, Git!')" > hello_git.py

git add hello_git.py

git commit -m "Initial commit"

- Pushing to a Remote Repository:

git status

git push origin main

- Use the token from last slide with your GitHub username.

CONCLUSION

1. **Professional Development**
   - Continue using GitHub to build a professional portfolio and engage in open-source projects.
2. **Encouragement to Explore Further**
   - Apply what you've learned by working on personal or group projects.
   - Seek Help and Collaborate: Don't hesitate to seek help from the GitHub community and collaborate with peers.
   - Stay Curious: Always be curious and explore new features and best practices.
3. **Empowerment Through Knowledge**
   - Understanding and effectively using GitHub will empower you to manage your projects better, collaborate efficiently, and continuously learn and grow as a developer.

THANK YOU

Centre for Artificial Intelligence and Robotics

research.utm.my/cairo/

cairo@utm.my / trainer’s email